算法板子

Contents

1. 多项式与生成函数 3
   1.1. NTT 3
   1.2. 常系数齐次线性递推-bostan-mori 算法 3
2. 计算几何 4
   2.1. 函数 4
   2.2. 距离的转换 4
   2.3. pick 定理 4
   2.4. 凸包（andrew 算法） 5
   2.5. 扫描线 5
   2.6. 旋转卡壳 5
   2.7. 半平面交 6
   2.8. 随机增量法 6
   2.9. 反演变换 8
   2.10. Delaunay 三角剖分 10
   2.11. 二、三维计算几何 12
   2.12. 仿射变换 12
3. 数论 12
   3.1. 欧拉筛 12
   3.2. 倍数筛(求约数数量 O(nlogn)) 12
   3.3. 线性筛求范围的莫比乌斯函数 12
   3.4. 求多个数的欧拉函数 12
   3.5. 线性求逆元 13
   3.6. Exgcd 13
   3.7. Bsgs（gcd(a,p)==1） 13
   3.8. Excrt（扩展中国剩余定理） 13
   3.9. 拓展欧拉定理 13
   3.10. MILLER-ROBIN 素性检验 13
   3.11. Pollard Rho 算法--分解质因子 14
   3.12. 常见数论函数的迪利克雷卷积(待施工) 15
   3.13. 莫比乌斯反演的两种形式(待施工) 15
   3.14. 常见组合公式(待施工) 15
   3.15. 数论分块(待施工) 15
   3.16. 求逆元意义下的组合数 15
4. 图论 16
   4.1. dsu on tree 16
   4.2. 欧拉路径 16
   4.3. hall 定理 16
   4.4. 二分图最大匹配（匈牙利算法，稠密图且点数大于 1e4 不适用） 16
   4.5. 最小费用最大流（spfa） 17
   4.6. 最大流（dinic 算法） 17
   4.7. 有源汇最大流 18
   4.8. 费用流 18
   4.9. 点分治 19
5. 数据结构 21
   5.1. 线段树 21
   5.2. 树状数组 21
   5.3. bitset 22
   5.4. 可删除并查集 22
   5.5. 关于 dsu 的一些碎碎念 23
   5.6. 带权并查集 23
   5.7. AC 自动机 24
   5.8. 笛卡尔树(区间最值,区间贡献) 25
   5.9. 虚树 25
6. 马拉车 27

7. 拉格朗日插值（通用版，$o(n^2)$） 27
8. 拉格朗日插值（连续整数） 28
9. Odt 28
10. 前缀数组（来自 oiwiki） 28
11. kmp（来自 oiwiki） 28
12. mt19937 28
13. 线性基 29
    13.1. 异或线性基典题 29
14. 强连通分量-tarjan 30
15. polya 定理 31
16. 求割点 31
17. 自定义 hash 31
18. Zobrist Hashing（xor hashing） 32
19. Lemma (Bertrand's postulate) 32
20. 半开半闭区间二分 32
21. 开区间二分(l,r) 32
22. 数位 dp 32
23. 关于 gcd 的常数优化 33
24. 高精度 34
25. 半开半闭区间线段树 36
26. 杂项 36
    26.1. min - max 恒等式(将 min 问题转化为 max 问题) 36
    26.2. 对区间离线操作的差分处理 36
    26.3. 对拍 36
    26.4. 一些常见的__builin_函数 37
    26.5. 表达式求值 37
    26.6. 优先队列自定义结构体比较 38
    26.7. __int128 输出(cout 实现) 38
    26.8. 三分 38
    26.9. GF(2)高斯消元判矩阵满秩 39
    26.10. sos dp 39
    26.11. CDQ 分治 39
    26.12. pb_ds 41
    26.13. floyd 判圈法 42
    26.14. st 表上二分(待施工) 43
    26.15. 向上取整与向下取整 43
    26.16. 随手记 43
    26.17. 碎碎念 43
    26.18. 一些思维(废话) 43

1. 多项式与生成函数

1.1. NTT

```
const int MOD = 998244353, G = 3; // 常见 NTT 模数及其原根
//power是ksm
void ntt(vector<int>& a, bool inv) {
    int n = a.size();
    for (int i = 1, j = 0; i<n; i++) {
        int bit = n>>1;
        for (; j&bit; bit>>= 1) j ^= bit;
        j ^= bit;
        if (i<j) swap(a[i], a[j]);
    }
    for (int len = 2; len<= n; len<<= 1) {
        ll wlen = power(G, (MOD - 1) / len);
        if (inv) wlen = power(wlen, MOD - 2);
        for (int i = 0; i<n; i += len) {
            ll w = 1;
            for (int j = 0; j<len / 2; j++) {
                int u = a[i + j], v = (int)(1LL * a[i + j + len / 2] * w % MOD);
                a[i + j] = (u + v) % MOD;
                a[i + j + len / 2] = (u - v + MOD) % MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (inv) {
        ll n_inv = power(n, MOD - 2);
        for (int&x : a) x = (int)(1LL * x * n_inv % MOD);
    }
}
vector<int>multiply(vector<int>a, vector<int>b)
{
    int n = 1, total = a.size() + b.size() - 1;
    while (n<total) n<<= 1;
    a.resize(n); b.resize(n);
    ntt(a, 0); ntt(b, 0);
    for (int i = 0; i<n; i++) a[i] = 1LL * a[i] * b[i] % MOD;
    ntt(a, 1);
    a.resize(total);
    //求循环卷积return res
    /*vector<int>res(L, 0);
    for (int i = 0; i<n; i++) {
        res[i % L] = (res[i % L] + a[i]) % MOD;
    }*/
    return a;}
```

1.2. 常系数齐次线性递推-bostan-mori 算法

求一个满足 k 阶齐次线性递推数列 a_i 的第 n 项，即：

$$a_n = \sum_{i=1}^{k} f_i \times a_{n-i}$$

输入格式
第一行两个数 n, k，如题面所述。
第二行 k 个数，表示 f
第三行 k 个数，表示 a

输出格式
一个数,表示$a_n \bmod 998244353$ 的值

```
i64 Bostan_mori(vector<i64> &P, vector<i64> &Q, int n) {
    vector<i64> Qm, A, B;
    while(n) {
        Qm = Q;
        for (int i = 1; i < Qm.size(); i += 2) {
            Qm[i] = (mod - Qm[i]) % mod;
        }
        A = mul(P, Qm);
        B = mul(Q, Qm);
        int bit = n & 1;
        vector<i64> nP, nQ;
        for (int i = bit; i < A.size(); i += 2) {
            nP.emplace_back(A[i]);
        }
        for (int i = 0; i < B.size(); i += 2) {
            nQ.emplace_back(B[i]);
        }
        P = nP;
        Q = nQ;
        n >>= 1;
    }
    return P[0] * inv(Q[0]) % mod;
}
void solve() {
    int n, k;
    cin >> n >> k;
    vector<int> f(k);
    for (auto &i : f) {
        cin >> i;
        i = (i % mod + mod) % mod;
    }
    vector<int> a(k);
    for (auto &i : a) {
        cin >> i;
        i = (i % mod + mod) % mod;
    }
    vector<i64> Q(k + 1);
    Q[0] = 1;
    for (int i = 1; i < k + 1; i++) {
        Q[i] = (mod - f[i - 1] % mod) % mod;
    }
    vector<i64> P(k);
    P[0] = (a[0] % mod + mod) % mod;
    for (int i = 1; i < k; i++) {
```

```
        P[i] = (a[i] % mod + mod) % mod;
        for (int j = 0; j < i; j++) {
            P[i] = (P[i] - 1ll * a[j] * f[i - j - 1] % mod +
mod) % mod;
        }
    }
    cout << Bostan_mori(P, Q, n) << '\n';
    return ;
}
```

Problem Statement
You are given a sequence of positive integers $A = (A_1, A_2, ..., A_N)$ of length N, and positive integers S and T.

Find the number of sequences of non-negative integers $(c_1, c_2, ..., c_N)$ that satisfy all of the following conditions, modulo 998244353:

$$c_1 + c_2 + \cdots + c_N = S$$

$$A_1 c_1 + A_2 c_2 + \cdots + A_N c_N = T$$

Constraints
$1 \leq N \leq 20$
$1 \leq S \leq 10^{18}$
$1 \leq T \leq 10^{18}$
$1 \leq A_i \leq 200$
$\sum_{i=1}^{N} A_i \leq 200$
All input values are integers

```
//像dp模拟bostan-mori的过程,实现也简单,处理方式很巧妙  gpt的说法:如果分母可以分解成容易处理的稀疏因子，并且经过“共轭相乘、指数变成偶数、指数除以 2”后，分母结构和分子规模仍然可控，就可以仿照这道题实现特殊化的 Bostan–Mori。
#include<bits/stdc++.h>
using namespace std;
using i64 = long long;
const int mod = 998244353;
void mol(i64 &x) {
    x %= mod;
    if (x < 0) x += mod;
    return ;
}
void solve() {
    i64 n, S, T;
    cin >> n >> S >> T;
    vector<int> a(n);
    int m = 0;
    for (auto &i : a) {
        cin >> i;
        m += i;
    }
    vector<vector<i64>> dp(2 * n + 1,
vector<i64>(2 * m + 1));
    vector<vector<i64>> ndp;
    dp[0][0] = 1;
    while(S | T) {
        ndp = dp;
        for (auto i : a) {
            for (int j = n * 2; j >= 1; j--) {
                for (int k = m * 2; k >= i; k--) {
                    ndp[j][k] += ndp[j - 1][k - i];
                    mol(ndp[j][k]);
                }
            }
        }
        int s = S & 1, t = T & 1;
        for (int i = 0; (i << 1 | s) <= n * 2; i++) {
            for (int j = 0; (j << 1 | t) <= m * 2; j++) {
                dp[i][j] = ndp[i << 1 | s][j << 1 | t];
                mol(dp[i][j]);
            }
        }
        S >>= 1;
        T >>= 1;
    }
    cout << dp[0][0] << '\n';
    return ;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
    return 0;
}
```

2. 计算几何

2.1. 函数

```
hypotl(x, y) ->sqrt(x * x + y * y)
acosl(ld) asinl(ld) atanl(ld) atan2l(y, x) fmodl(x,
mod)
pi = acosl(-1.0L)
/*约束到[0,2*pi]*/
i = fmodl(i, 2 * pi);
if (i < 0) i += 2 * pi;
```

2.2. 距离的转换

$|x_1 - x_2| + |y_1 - y_2| = \max(|(x_1 + y_1) - (x_2 + y_2)|, |(x_1 - y_1) - (x_2 - y_2)|)$

2.3. pick 定理

给定顶点均为整点的简单多边形，皮克定理说明了其面积 A和内部格点数目 i、边上格点数目 b的关系

$A = i + \frac{b}{2} - 1$

2.4. 凸包（andrew 算法）

```
struct Point {
    long long x, y;
    // 排序：先按 x 升序，再按 y 升序
    bool operator<(const Point& t) const {
        return x == t.x ? y<t.y : x<t.x;
    }
};
// 叉积：(B-A) x (C-A)。>0 表示 C 在 AB 左侧（逆时针旋转）
long long cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}
vector<Point>getConvexHull(vector<Point>&p) {
    int n = p.size(), k = 0;
    if (n <= 2) return p;
    vector<Point>res(2 * n);
    sort(p.begin(), p.end());
    // 求下凸壳
    for (int i = 0; i<n; i++) {
        while (k>= 2&&cross(res[k - 2], res[k - 1], p[i])<= 0) k--;
        res[k++] = p[i];
    }
    // 求上凸壳
    for (int i = n - 2, t = k + 1; i>= 0; i--) {
        while (k>= t&&cross(res[k - 2], res[k - 1], p[i])<= 0) k--;
        res[k++] = p[i];
    }
    res.resize(k - 1); // 最后一个点和第一个点重合，删掉
    return res;
}
```

2.4.1. 闵可夫斯基和

```
template <class T>
struct Point {
  T x, y;

  Point(T x = 0, T y = 0) : x(x), y(y) {}

  friend Point operator+(const Point &a, const Point &b) {
    return {a.x + b.x, a.y + b.y};
  }

  friend Point operator-(const Point &a, const Point &b) {
    return {a.x - b.x, a.y - b.y};
  }

  // 点乘
  friend T operator*(const Point &a, const Point &b) {
    return a.x * b.x + a.y * b.y;
  }

  // 叉乘
  friend T operator^(const Point &a, const Point &b) {
    return a.x * b.y - a.y * b.x;
  }
};

template <class T>
vector<Point<T>>
minkowski_sum(vector<Point<T>> a, vector<Point<T>> b) {
  vector<Point<T>> c{a[0] + b[0]};
  for (usz i = 0; i + 1 < a.size(); ++i) a[i] = a[i + 1] - a[i];
  for (usz i = 0; i + 1 < b.size(); ++i) b[i] = b[i + 1] - b[i];
  a.pop_back(), b.pop_back();
  c.resize(a.size() + b.size() + 1);
  merge(a.begin(), a.end(), b.begin(), b.end(), c.begin() + 1,
        [](const Point<T> &a, const Point<T> &b) { return (a ^ b) > 0; });
  for (usz i = 1; i < c.size(); ++i) c[i] = c[i] + c[i - 1];
  return c;
}
```

2.5. 扫描线

2.6. 旋转卡壳

```
int sta[N], top; // 将凸包上的节点编号存在栈里，第一个和最后一个节点编号相同

ll pf(ll x) { return x * x; }

ll dis(int p, int q) { return pf(a[p].x - a[q].x) + pf(a[p].y - a[q].y); }

ll sqr(int p, int q, int y) { return abs((a[q] - a[p]) * (a[y] - a[q])); }

ll mx;

void get_longest() { // 求凸包直径
  int j = 3;
  if (top < 4) {
    mx = dis(sta[1], sta[2]);
    return;
  }
  for (int i = 1; i < top; ++i) {
```

```
    while (sqr(sta[i], sta[i + 1], sta[j]) <=
           sqr(sta[i], sta[i + 1], sta[j % top + 1]))
      j = j % top + 1;
    mx = max(mx, max(dis(sta[i + 1], sta[j]),
dis(sta[i], sta[j])));
  }
}
```

2.7. 半平面交

```
friend bool operator<(seg x, seg y) {
  db t1 = atan2((x.b - x.a).y, (x.b - x.a).x);
  db t2 = atan2((y.b - y.a).y, (y.b - y.a).x);  // 求极角
  if (fabs(t1 - t2) > eps)                      // 如果极角不等
    return t1 < t2;
  return (y.a - x.a) * (y.b - x.a) >
         eps;  // 判断向量x在y的哪边，令最靠左的排在最左边
}
// pnt its(seg a,seg b)表示求线段a,b的交点
// s[]是极角排序后的向量
// q[]是向量队列
// t[i]是s[i-1]与s[i]的交点
// 【码风】队列的范围是(l,r]
// 求的是向量左侧的半平面
int l = 0, r = 0;
for (int i = 1; i <= n; ++i)
  if (s[i] != s[i - 1]) {
    // 注意要先检查队尾
    while (r - l > 1 && (s[i].b - t[r]) * (s[i].a - t[r]) >
                        eps)  // 如果上一个交点在向量右侧则弹出队尾
      --r;
    while (r - l > 1 && (s[i].b - t[l + 2]) * (s[i].a - t[l + 2]) >
                        eps)  // 如果第一个交点在向量右侧则弹出队首
      ++l;
    q[++r] = s[i];
    if (r - l > 1) t[r] = its(q[r], q[r - 1]);  // 求新交点
  }
while (r - l > 1 &&
       (q[l + 1].b - t[r]) * (q[l + 1].a - t[r]) > eps)  // 注意删除多余元素
  --r;
t[r + 1] = its(q[l + 1], q[r]);  // 再求出新的交点
++r;
// 这里不能在t里面++r需要注意一下.....
```

2.8. 随机增量法

```
/*
  最小圆覆盖问题:在一个平面上有 n 个点，求一个半径最小的圆，能覆盖所有的点.
  下面这个做法用随机化打乱后期望o(n),证明较为复杂，下面给了两种写法(其实一样),第二种是我自己写的,感觉代码会更清晰点
*/
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <iostream>

using namespace std;

int n;
double r;

struct point {
  double x, y;
} p[100005], o;

double sqr(double x) { return x * x; }

double dis(point a, point b) { return sqrt(sqr(a.x - b.x) + sqr(a.y - b.y)); }

bool cmp(double a, double b) { return fabs(a - b) < 1e-8; }

point geto(point a, point b, point c) {
  double a1, a2, b1, b2, c1, c2;
  point ans;
  a1 = 2 * (b.x - a.x), b1 = 2 * (b.y - a.y),
  c1 = sqr(b.x) - sqr(a.x) + sqr(b.y) - sqr(a.y);
  a2 = 2 * (c.x - a.x), b2 = 2 * (c.y - a.y),
  c2 = sqr(c.x) - sqr(a.x) + sqr(c.y) - sqr(a.y);
  if (cmp(a1, 0)) {
    ans.y = c1 / b1;
    ans.x = (c2 - ans.y * b2) / a2;
  } else if (cmp(b1, 0)) {
    ans.x = c1 / a1;
    ans.y = (c2 - ans.x * a2) / b2;
  } else {
    ans.x = (c2 * b1 - c1 * b2) / (a2 * b1 - a1 * b2);
    ans.y = (c2 * a1 - c1 * a2) / (b2 * a1 - b1 * a2);
  }
  return ans;
}

int main() {
  scanf("%d", &n);
  for (int i = 1; i <= n; i++) scanf("%lf%lf", &p[i].x, &p[i].y);
```

```
  for (int i = 1; i <= n; i++) swap(p[rand() % n + 1],
p[rand() % n + 1]);
  o = p[1];
  for (int i = 1; i <= n; i++) {
    if (dis(o, p[i]) < r || cmp(dis(o, p[i]), r))
continue;
    o.x = (p[i].x + p[1].x) / 2;
    o.y = (p[i].y + p[1].y) / 2;
    r = dis(p[i], p[1]) / 2;
    for (int j = 2; j < i; j++) {
      if (dis(o, p[j]) < r || cmp(dis(o, p[j]), r))
continue;
      o.x = (p[i].x + p[j].x) / 2;
      o.y = (p[i].y + p[j].y) / 2;
      r = dis(p[i], p[j]) / 2;
      for (int k = 1; k < j; k++) {
        if (dis(o, p[k]) < r || cmp(dis(o, p[k]), r))
continue;
        o = geto(p[i], p[j], p[k]);
        r = dis(o, p[i]);
      }
    }
  }
  printf("%.10lf\n%.10lf %.10lf", r, o.x, o.y);
  return 0;
}
```

```
/*
   自己写的最小圆覆盖代码，有点丑
*/
#include<bits/stdc++.h>
using namespace std;
const double eps = 1e-9;
mt19937
rng(chrono::steady_clock::now().time_since_epoch().count());
void solve() {
    int n;
    cin >> n;
    vector<pair<double,double>> a(n);
    for (auto &[x, y] : a) {
        cin >> x >> y;
    }
    shuffle(a.begin(), a.end(), rng);
    pair<double,double> c = a[0];
    double r = 0;
    auto dis = [&](pair<double,double> &a) -
>double {
        auto &[x, y] = a;
        auto &[x1, y1] = c;
        return sqrt((x - x1) * (x - x1) + (y - y1) * (y -
y1));
    };
    auto get_v = [&](vector<array<double,2>> &x)
->double {
        auto [a1, b1] = x[0];
        auto [a2, b2] = x[1];
        return a1 * b2 - a2 * b1;
    };
    auto get_c = [&](int i, int j, int k) -
>pair<double,double>{
        auto &[xi, yi] = a[i];
        auto &[xj, yj] = a[j];
        auto &[xk, yk] = a[k];
        vector<array<double,2>> mk(2), mk_x(2),
mk_y(2);
        mk[0] = {-2 * xi + 2 * xj, -2 * yi + 2 * yj};
        mk[1] = {-2 * xi + 2 * xk, -2 * yi + 2 * yk};
        double base = get_v(mk);
        mk_x[0] = {xj * xj - xi * xi + yj * yj - yi * yi,
-2 * yi + 2 * yj};
        mk_x[1] = {xk * xk - xi * xi + yk * yk - yi *
yi, -2 * yi + 2 * yk};
        double x = get_v(mk_x) / base;
        mk_y[0] = {-2 * xi + 2 * xj, xj * xj - xi * xi +
yj * yj - yi * yi};
        mk_y[1] = {-2 * xi + 2 * xk, xk * xk - xi * xi +
yk * yk - yi * yi};
        double y = get_v(mk_y) / base;
        return {x, y};
    };
    for (int i = 1; i < n; i++) {
        if (dis(a[i]) > r + eps) {
            c = a[i];
            r = 0;
            for (int j = 0; j < i; j++) {
                if (dis(a[j]) > r + eps) {
                    auto [x, y] = a[i];
                    auto [x1, y1] = a[j];
                    c = pair{(x + x1) / 2, (y + y1) / 2};
                    r = dis(a[j]);
                    for (int k = 0; k < j; k++) {
                        if (dis(a[k]) > r + eps) {
                            auto [x1, y1] = pair{a[k].first -
a[i].first, a[k].second - a[i].second};
                            auto [x2, y2] = pair{a[j].first -
a[i].first, a[j].second - a[i].second};
                            if (fabs(x1 * y2 - x2 * y1) <eps)
continue;
                            c = get_c(i, j, k);
                            r = dis(a[k]);
                        }
                    }
                }
            }
        }
    }
    cout << fixed << setprecision(9) << r << '\n' <<
c.first << ' ' << c.second << '\n';
    return ;
}
int main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
    return 0;
}
```

2.9. 反演变换

定义:
给定反演中心点 O和反演半径 R,若平面上点 和 P' 满足:
(1)点 P' 在射线 OP 上
(2)$|OP| \cdot |OP'| = R^2$
则称点 P和点 P'互为反演点.

性质:
(1)除反演中心外，平面上的每一个点都只有唯一的反演点，且这种关系是对称的，位于反演圆上的点，保持在原处，位于反演圆外部的点，变为圆内部的点，位于反演圆内部的点，变为圆外部的点。
(2)任意一条不过反演中心的直线，它的反形是经过反演中心的圆，反之亦然，特别地，过反演中心相交的圆，变为不过反演中心的相交直线
(3)反演不改变相切、平行、相交的关系
记圆 A半径为 $r1$,其反演图形圆 B半径为 $r2$,则有:

$$r_2 = \frac{1}{2}\left(\frac{1}{|OA| - r_1} - \frac{1}{|OA| + r_1}\right)R^2$$

记点 O 坐标为 (x_0, y_0)，点 A 坐标为 (x_1, y_1)，点 B 坐标为 (x_2, y_2)，则有:

$$x_2 = x_0 + \frac{|OB|}{|OA|}(x_1 - x_0)$$

$$y_2 = y_0 + \frac{|OB|}{|OA|}(y_1 - y_0)$$

```
/*
题目:求过两圆外一点，且与两圆相切的所有的圆.
*/
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstring>
#include <iostream>
#include <vector>
using namespace std;

constexpr double EPS = 1e-8;  // 精度系数
const double PI = acos(-1.0);  // π
constexpr int N = 4;

// 点的定义
struct Point {
  double x, y;
  Point(double x = 0, double y = 0) : x(x), y(y) {}

  bool operator<(Point A) const { return x == A.x ? y < A.y : x < A.x; }
};

// 向量的定义
using Vector = Point;

// 向量加法
Vector operator+(Vector A, Vector B) { return Vector(A.x + B.x, A.y + B.y); }

// 向量减法
Vector operator-(Vector A, Vector B) { return Vector(A.x - B.x, A.y - B.y); }

// 向量数乘
Vector operator*(Vector A, double p) { return Vector(A.x * p, A.y * p); }

// 向量数除
Vector operator/(Vector A, double p) { return Vector(A.x / p, A.y / p); }

// 与0的关系
int dcmp(double x) {
  if (fabs(x) < EPS) return 0;
  return x < 0 ? -1 : 1;
}

// 向量点乘
double Dot(Vector A, Vector B) { return A.x * B.x + A.y * B.y; }

// 向量长度
double Length(Vector A) { return sqrt(Dot(A, A)); }

// 向量叉乘
double Cross(Vector A, Vector B) { return A.x * B.y - A.y * B.x; }

// 点在直线上投影
Point GetLineProjection(Point P, Point A, Point B) {
  Vector v = B - A;
  return A + v * (Dot(v, P - A) / Dot(v, v));
}

// 圆
struct Circle {
  Point c;
  double r;
```

```
  Circle() : c(Point(0, 0)), r(0) {}

  Circle(Point c, double r = 0) : c(c), r(r) {}

  // 输入极角返回点坐标
  Point point(double a) { return Point(c.x + cos(a)
* r, c.y + sin(a) * r); }
};

// 两圆公切线 返回切线的条数，-1表示无穷多条切线
// a[i] 和 b[i] 分别是第i条切线在圆A和圆B上的切点
int getTangents(Circle A, Circle B, Point* a,
Point* b) {
  int cnt = 0;
  if (A.r < B.r) {
    swap(A, B);
    swap(a, b);
  }
  double d2 =
      (A.c.x - B.c.x) * (A.c.x - B.c.x) + (A.c.y - B.c.y)
* (A.c.y - B.c.y);
  double rdiff = A.r - B.r;
  double rsum = A.r + B.r;
  if (dcmp(d2 - rdiff * rdiff) < 0) return 0;  // 内含

  double base = atan2(B.c.y - A.c.y, B.c.x - A.c.x);
  if (dcmp(d2) == 0 && dcmp(A.r - B.r) == 0)
return -1;  // 无限多条切线
  if (dcmp(d2 - rdiff * rdiff) == 0) {  // 内切，一条切线
    a[cnt] = A.point(base);
    b[cnt] = B.point(base);
    ++cnt;
    return 1;
  }
  // 有外公切线
  double ang = acos(rdiff / sqrt(d2));
  a[cnt] = A.point(base + ang);
  b[cnt] = B.point(base + ang);
  ++cnt;
  a[cnt] = A.point(base - ang);
  b[cnt] = B.point(base - ang);
  ++cnt;
  if (dcmp(d2 - rsum * rsum) == 0) {  // 一条内公切线
    a[cnt] = A.point(base);
    b[cnt] = B.point(PI + base);
    ++cnt;
  } else if (dcmp(d2 - rsum * rsum) > 0) {  // 两条内公切线
    double ang = acos(rsum / sqrt(d2));
    a[cnt] = A.point(base + ang);
    b[cnt] = B.point(PI + base + ang);
    ++cnt;
    a[cnt] = A.point(base - ang);
    b[cnt] = B.point(PI + base - ang);
    ++cnt;
  }
  return cnt;
}

// 点 O 在圆 A 外，求圆 A 的反演圆 B，R 是反演半径
Circle Inversion_C2C(Point O, double R, Circle
A) {
  double OA = Length(A.c - O);
  double RB = 0.5 * ((1 / (OA - A.r)) - (1 / (OA +
A.r))) * R * R;
  double OB = OA * RB / A.r;
  double Bx = O.x + (A.c.x - O.x) * OB / OA;
  double By = O.y + (A.c.y - O.y) * OB / OA;
  return Circle(Point(Bx, By), RB);
}

// 直线反演为过 O 点的圆 B，R 是反演半径
Circle Inversion_L2C(Point O, double R, Point A,
Vector v) {
  Point P = GetLineProjection(O, A, A + v);
  double d = Length(O - P);
  double RB = R * R / (2 * d);
  Vector VB = (P - O) / d * RB;
  return Circle(O + VB, RB);
}

// 返回 true 如果 A B 两点在直线同侧
bool theSameSideOfLine(Point A, Point B, Point
S, Vector v) {
  return dcmp(Cross(A - S, v)) * dcmp(Cross(B - S,
v)) > 0;
}

int main() {
  int T;
  scanf("%d", &T);
  while (T--) {
    Circle A, B;
    Point P;
    scanf("%lf%lf%lf", &A.c.x, &A.c.y, &A.r);
    scanf("%lf%lf%lf", &B.c.x, &B.c.y, &B.r);
    scanf("%lf%lf", &P.x, &P.y);
    Circle NA = Inversion_C2C(P, 10, A);
    Circle NB = Inversion_C2C(P, 10, B);
    Point LA[N], LB[N];
    Circle ansC[N];
    int q = getTangents(NA, NB, LA, LB), ans = 0;
    for (int i = 0; i < q; ++i)
      if (theSameSideOfLine(NA.c, NB.c, LA[i],
LB[i] - LA[i])) {
```

```
      if (!theSameSideOfLine(P, NA.c, LA[i], LB[i]
- LA[i])) continue;
      ansC[ans++] = Inversion_L2C(P, 10, LA[i],
LB[i] - LA[i]);
    }
   printf("%d\n", ans);
   for (int i = 0; i < ans; ++i) {
    printf("%.8f %.8f %.8f\n", ansC[i].c.x,
ansC[i].c.y, ansC[i].r);
   }
 }

 return 0;
}
```

2.10. Delaunay 三角剖分

```
#include <algorithm>
#include <cmath>
#include <cstring>
#include <list>
#include <utility>
#include <vector>

constexpr double EPS = 1e-8;
constexpr int MAXV = 10000;

struct Point {
 double x, y;
 int id;

 Point(double a = 0, double b = 0, int c = -1) :
x(a), y(b), id(c) {}

 bool operator<(const Point &a) const {
  return x < a.x || (fabs(x - a.x) < EPS && y <
a.y);
 }

 bool operator==(const Point &a) const {
  return fabs(x - a.x) < EPS && fabs(y - a.y) <
EPS;
 }

 double dist2(const Point &b) {
  return (x - b.x) * (x - b.x) + (y - b.y) * (y - b.y);
 }
};

struct Point3D {
 double x, y, z;

 Point3D(double a = 0, double b = 0, double c =
0) : x(a), y(b), z(c) {}

 Point3D(const Point &p) { x = p.x, y = p.y, z =
p.x * p.x + p.y * p.y; }

 Point3D operator-(const Point3D &a) const {
  return Point3D(x - a.x, y - a.y, z - a.z);
 }

 double dot(const Point3D &a) { return x * a.x +
y * a.y + z * a.z; }
};

struct Edge {
 int id;
 std::list<Edge>::iterator c;

 Edge(int id = 0) { this->id = id; }
};

int cmp(double v) { return fabs(v) > EPS ? (v > 0 ?
1 : -1) : 0; }

double cross(const Point &o, const Point &a,
const Point &b) {
 return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x -
o.x);
}

Point3D cross(const Point3D &a, const Point3D
&b) {
 return Point3D(a.y * b.z - a.z * b.y, -a.x * b.z +
a.z * b.x,
                a.x * b.y - a.y * b.x);
}

int inCircle(const Point &a, Point b, Point c,
const Point &p) {
 if (cross(a, b, c) < 0) std::swap(b, c);
 Point3D a3(a), b3(b), c3(c), p3(p);
 b3 = b3 - a3, c3 = c3 - a3, p3 = p3 - a3;
 Point3D f = cross(b3, c3);
 return cmp(p3.dot(f));  // check same direction,
in: < 0, on: = 0, out: > 0
}

int intersection(const Point &a, const Point &b,
const Point &c,
                 const Point &d) {  // seg(a, b) and seg(c,
d)
 return cmp(cross(a, c, b)) * cmp(cross(a, b, d)) >
0 &&
        cmp(cross(c, a, d)) * cmp(cross(c, d, b)) > 0;
}

class Delaunay {
 public:
 std::list<Edge> head[MAXV];  // graph
 Point p[MAXV];
```

```
  int n, rename[MAXV];

  void init(int n, Point p[]) {
    memcpy(this->p, p, sizeof(Point) * n);
    std::sort(this->p, this->p + n);
    for (int i = 0; i < n; i++) rename[p[i].id] = i;
    this->n = n;
    divide(0, n - 1);
  }

  void addEdge(int u, int v) {
    head[u].push_front(Edge(v));
    head[v].push_front(Edge(u));
    head[u].begin()->c = head[v].begin();
    head[v].begin()->c = head[u].begin();
  }

  void divide(int l, int r) {
    if (r - l <= 2) {  // #point <= 3
      for (int i = l; i <= r; i++)
        for (int j = i + 1; j <= r; j++) addEdge(i, j);
      return;
    }
    int mid = (l + r) / 2;
    divide(l, mid);
    divide(mid + 1, r);

    std::list<Edge>::iterator it;
    int nowl = l, nowr = r;

    for (int update = 1; update;) {
      // find left and right convex, lower common
tangent
      update = 0;
      Point ptL = p[nowl], ptR = p[nowr];
      for (it = head[nowl].begin(); it !=
head[nowl].end(); it++) {
        Point t = p[it->id];
        double v = cross(ptR, ptL, t);
        if (cmp(v) > 0 || (cmp(v) == 0 && ptR.dist2(t)
< ptR.dist2(ptL))) {
          nowl = it->id, update = 1;
          break;
        }
      }
      if (update) continue;
      for (it = head[nowr].begin(); it !=
head[nowr].end(); it++) {
        Point t = p[it->id];
        double v = cross(ptL, ptR, t);
        if (cmp(v) < 0 || (cmp(v) == 0 && ptL.dist2(t)
< ptL.dist2(ptR))) {
          nowr = it->id, update = 1;
          break;
        }
      }
    }

    addEdge(nowl, nowr);  // add tangent

    for (int update = 1; true;) {
      update = 0;
      Point ptL = p[nowl], ptR = p[nowr];
      int ch = -1, side = 0;
      for (it = head[nowl].begin(); it !=
head[nowl].end(); it++) {
        if (cmp(cross(ptL, ptR, p[it->id])) > 0 &&
            (ch == -1 || inCircle(ptL, ptR, p[ch], p[it-
>id]) < 0)) {
          ch = it->id, side = -1;
        }
      }
      for (it = head[nowr].begin(); it !=
head[nowr].end(); it++) {
        if (cmp(cross(ptR, p[it->id], ptL)) > 0 &&
            (ch == -1 || inCircle(ptL, ptR, p[ch], p[it-
>id]) < 0)) {
          ch = it->id, side = 1;
        }
      }
      if (ch == -1) break;  // upper common tangent
      if (side == -1) {
        for (it = head[nowl].begin(); it !=
head[nowl].end();) {
          if (intersection(ptL, p[it->id], ptR, p[ch])) {
            head[it->id].erase(it->c);
            head[nowl].erase(it++);
          } else {
            it++;
          }
        }
        nowl = ch;
        addEdge(nowl, nowr);
      } else {
        for (it = head[nowr].begin(); it !=
head[nowr].end();) {
          if (intersection(ptR, p[it->id], ptL, p[ch])) {
            head[it->id].erase(it->c);
            head[nowr].erase(it++);
          } else {
            it++;
          }
        }
        nowr = ch;
        addEdge(nowl, nowr);
      }
    }
  }

  std::vector<std::pair<int, int>> getEdge() {
    std::vector<std::pair<int, int>> ret;
    ret.reserve(n);
```

```
    std::list<Edge>::iterator it;
    for (int i = 0; i < n; i++) {
      for (it = head[i].begin(); it != head[i].end(); it++) {
        if (it->id < i) continue;
        ret.push_back(std::make_pair(p[i].id, p[it->id].id));
      }
    }
    return ret;
  }
};
```

2.11. 二、三维计算几何

https://oi-wiki.org/geometry/2d/
https://oi-wiki.org/geometry/3d/

2.12. 仿射变换

3. 数论

3.1. 欧拉筛

```
vector<int> Euler(int n) {
    vector<int> not_prime(n + 1, 0);
    vector<int> prime;
    for (int i = 2; i <= n; i++) {
        if (!not_prime[i]) {
            prime.emplace_back(i);
        }
        for (auto j : prime) {
            if (1ll * i * j > n) break;
            not_prime[i * j] = 1;
            if (i % j == 0) break;
        }
    }
    return prime;
}
```

3.2. 倍数筛(求约数数量 O(nlogn))

```
vector<vector<int>> divisors(N + 1);

for (int d = 1; d <= N; ++d) {
    for (int x = d; x <= N; x += d) {
        divisors[x].push_back(d);
    }
}
```

3.3. 线性筛求范围的莫比乌斯函数

```
vector<int> primes;
vector<int> mu;
vector<char> composite;


void init_mu(int N) {
    mu.assign(N + 1, 0);
    composite.assign(N + 1, false);

    mu[1] = 1;

    for (int i = 2; i <= N; ++i) {
        if (!composite[i]) {
            primes.push_back(i);
            mu[i] = -1;
        }

        for (int p : primes) {
            if (1LL * i * p > N) break;

            composite[i * p] = true;

            if (i % p == 0) {
                // p^2 | i * p
                mu[i * p] = 0;
                break;
            } else {
                // 增加了一个不同的质因子 p
                mu[i * p] = -mu[i];
            }
        }
    }
}
```

3.4. 求多个数的欧拉函数

```
vector<int> cal_euler(int x) {
    vector<int> re(x + 1);
    vector<int> prime;
    vector<int> not_prime(x + 1, 0);
    for (int i = 2; i <= x; i++) {
        if (!not_prime[i]) {
            prime.emplace_back(i);
            re[i] = i - 1;
        }
        for (auto j : prime) {
            if (i * j > x) break;
            not_prime[i * j] = 1;
            if (i % j == 0) {
                re[i * j] = re[i] * j;
            }
            else {
                re[i * j] = re[i] * re[j];
            }
            if (i % j == 0) break;
        }
    }
    return re;
}
```

3.5. 线性求逆元

```
inv_num[1] = 1;
for (int i = 2; i < limit; i++) {
    inv_num[i] = (mod - mod / i) * inv_num[mod
% i] % mod;
}
```

3.6. Exgcd

```
i64 exgcd(i64 a, i64 b, i64&x, i64&y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    i64 g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
```

3.7. Bsgs（gcd(a,p)==1）

```
ll bsgs(ll a, ll b, ll p) {
    if (1 % p == b % p) return 0;
    unordered_map<ll, ll>hash;
    ll m = ceil(sqrt(p));

    // Baby-step: 预处理 a^j * b 并存入哈希表
    ll cur = b % p;
    for (int j = 0; j<m; j++) {
        hash[cur] = j;
        cur = cur * a % p;
    }

    // Giant-step: 计算 am = a^m，然后枚举 i 检
查 am^i
    ll am = 1;
    for (int i = 0; i<m; i++) am = am * a % p;

    cur = am;
    for (int i = 1; i<= m; i++) {
        if (hash.count(cur)) return i * m - hash[cur];
        cur = cur * am % p;
    }
    return -1; // 无解
}
```

3.8. Excrt（扩展中国剩余定理）

```
// 这里的乘法取模是为了防止 (a * b) % m 溢出，
如果 a, b 较小可直接使用 a * b % m
ll qmul(ll a, ll b, ll m) {
    ll res = 0;
    while (b) {
        if (b&1) res = (res + a) % m;
        a = (a + a) % m;
        b>>= 1;
    }
    return res;
}
// n 个方程：x = r[i] (mod m[i])
ll excrt(vector<ll>&m, vector<ll>&r) {
    ll M = m[0], R = r[0]; // R 为当前的解，M 为
当前的最小公倍数
    for (int i = 1; i<m.size(); i++) {
        ll x, y;
        ll d = exgcd(M, m[i], x, y);
        if ((r[i] - R) % d != 0) return -1; // 无解

        // 求解 M*x = r[i] - R (mod m[i])
        ll mod = m[i] / d;
        x = qmul((r[i] - R) / d, x, mod); // 这里的乘
法注意溢出
        if (x<0) x += mod;

        R += x * M;
        M = M / d * m[i];
        R = (R % M + M) % M;
    }
    return R;
}
```

3.9. 拓展欧拉定理

$$a^k \equiv \begin{cases} a^{k \bmod \varphi(m)} & \gcd(a,m)=1 \\ a^k & \gcd(a,m)\neq 1 \wedge k<\varphi(m) \\ a^{k \bmod \varphi(m)+\varphi(m)} & \gcd(a,m)\neq 1 \wedge k\geq \varphi(m) \end{cases} \pmod m$$

3.10. MILLER-ROBIN 素性检验

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 mul_mod(u64 a, u64 b, u64 mod) {
    return (u128)a * b % mod;
}
u64 pow_mod(u64 a, u64 b, u64 mod) {
    u64 res = 1;
    while (b) {
        if (b & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
    return res;
}
bool is_prime(u64 n) {
    if (n < 2) return false;
    static const u64 small_primes[] = {
```

```
        2, 3, 5, 7, 11, 13,
        17, 19, 23, 29, 31, 37
    };
    for (u64 p : small_primes) {
        if (n % p == 0) return n == p;
    }
    // n - 1 = d * 2^s
    u64 d = n - 1;
    int s = 0;
    while ((d & 1) == 0) {
        d >>= 1;
        ++s;
    }
    // 对所有 uint64_t 都成立的确定性底数
    static const u64 bases[] = {
        2, 325, 9375, 28178,
        450775, 9780504, 1795265022
    };
    for (u64 a : bases) {
        if (a % n == 0) continue;
        u64 x = pow_mod(a % n, d, n);
        if (x == 1 || x == n - 1) continue;
        bool passed = false;
        for (int r = 1; r < s; ++r) {
            x = mul_mod(x, x, n);
            if (x == n - 1) {
                passed = true;
                break;
            }
        }
        if (!passed) return false;
    }
    return true;
}
```

3.11. Pollard Rho 算法--分解质因子

```
using u64 = std::uint64_t;
using u128 = __uint128_t;

std::mt19937_64 rng(
std::chrono::steady_clock::now().time_since_epoch().count()
);

u64 mul_mod(u64 a, u64 b, u64 mod) {
    return static_cast<u128>(a) * b % mod;
}

u64 power_mod(u64 a, u64 exponent, u64 mod) {
    u64 result = 1;

    while (exponent > 0) {
        if (exponent & 1) {
            result = mul_mod(result, a, mod);
        }
        a = mul_mod(a, a, mod);
        exponent >>= 1;
    }

    return result;
}

// 对 uint64_t 范围确定正确的 Miller-Rabin。
bool is_prime(u64 n) {
    if (n < 2) {
        return false;
    }

    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL,
13ULL,
                  17ULL, 19ULL, 23ULL, 29ULL, 31ULL,
37ULL}) {
        if (n % p == 0) {
            return n == p;
        }
    }

    u64 d = n - 1;
    int s = 0;

    while ((d & 1) == 0) {
        d >>= 1;
        ++s;
    }

    constexpr u64 bases[] = {
        2ULL,
        325ULL,
        9375ULL,
        28178ULL,
        450775ULL,
        9780504ULL,
        1795265022ULL
    };

    for (u64 a : bases) {
        if (a % n == 0) {
            continue;
        }

        u64 x = power_mod(a % n, d, n);

        if (x == 1 || x == n - 1) {
            continue;
        }

        bool passed = false;

        for (int r = 1; r < s; ++r) {
            x = mul_mod(x, x, n);
```

```
            if (x == n - 1) {
                passed = true;
                break;
            }
        }

        if (!passed) {
            return false;
        }
    }

    return true;
}

u64 pollard_rho(u64 n) {
    if (n % 2 == 0) {
        return 2;
    }

    if (n % 3 == 0) {
        return 3;
    }

    while (true) {
        u64 x = rng() % (n - 2) + 2;
        u64 y = x;
        u64 c = rng() % (n - 1) + 1;
        u64 d = 1;

        auto next = [&](u64 value) -> u64 {
            return static_cast<u64>(
                (static_cast<u128>(mul_mod(value,
value, n)) + c) % n
            );
        };

        while (d == 1) {
            x = next(x);
            y = next(next(y));

            u64 difference = x > y ? x - y : y - x;
            d = std::gcd(difference, n);
        }

        if (d != n) {
            return d;
        }
    }
}

void factorize(u64 n, std::vector<u64>& factors) {
    if (n == 1) {
        return;
    }
    if (is_prime(n)) {
        factors.push_back(n);
        return;
    }

    u64 factor = pollard_rho(n);

    factorize(factor, factors);
    factorize(n / factor, factors);
}
```

3.12. 常见数论函数的迪利克雷卷积(待施工)

3.13. 莫比乌斯反演的两种形式(待施工)

3.14. 常见组合公式(待施工)

3.15. 数论分块(待施工)

```
for (long long l = L; l <= R; ) {
    long long q = n / l;
    long long r = min(R, n / q);

    // 处理 [l,r]。

    l = r + 1;
}
```

3.16. 求逆元意义下的组合数

```
using int64 = long long;
const int MOD = 1e9 + 7; // 必须是质数
const int N = 1e6;      // 最大 n，根据题目修改

int64 fac[N + 1], ifac[N + 1];

int64 qpow(int64 a, int64 b) {
    int64 res = 1;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

void init() {
    fac[0] = 1;
    for (int i = 1; i <= N; i++)
        fac[i] = fac[i - 1] * i % MOD;

    ifac[N] = qpow(fac[N], MOD - 2); // 费马小定
理求逆元
    for (int i = N; i >= 1; i--)
        ifac[i - 1] = ifac[i] * i % MOD;
}
```

```
int64 C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fac[n] * ifac[k] % MOD * ifac[n - k] %
MOD;
}
```

4. 图论

4.1. dsu on tree

```
void dfs_3(int p, int fa, bool keep) {
    for (auto i : e[p]) {
        if (i == fa || i == heavy[p]) continue;
        dfs_3(i, p, 0);
    }

    if (heavy[p] != -1) {
        dfs_3(heavy[p], p, 1);
    }

    //query(p, p);
    //add(p, 1);

    for (auto i : e[p]) {
        if (i == fa || i == heavy[p]) continue;

        // for (int j = tin[i]; j <= tout[i]; j++) {
        //     query(euler[j], p);
        // }

        // for (int j = tin[i]; j <= tout[i]; j++) {
        //     add(euler[j], +1);
        // }
    }
    if (!keep) {
        for (int i = tin[p]; i <= tout[p]; i++) {
            add(euler[i], -1);
        }
    }
    return ;
}
```

4.2. 欧拉路径

```
const int MAXN = 100005;
struct Edge {
    int to, id;
};
vector<Edge>adj[MAXN];
int head[MAXN]; // 当前弧优化：记录每个点
处理到了第几条边
bool used[MAXN * 2]; // 标记边是否被访问过
（处理无向图或多重边）
vector<int> path; // 存储最终路径（点序列或边
序列）

void dfs(int u) {
    // 遍历从 u 出发的边，head[u] 保证每条边
只被扫描一次
    for (int &i = head[u]; i<adj[u].size(); ) {
        Edge e = adj[u][i++]; // 注意这里必须先 i+
+ 再递归
        if (used[e.id]) continue;
        used[e.id] = true; // 标记该边已用
        dfs(e.to);
    }
    path.push_back(u); // 回溯时加入序列
}

// 得到路径：reverse(path.begin(), path.end());
```

4.3. hall 定理

- 设二分图 G = (L, R, E)，L 为左部，R 为右部。
- 1) 存在匹配覆盖 L 中所有顶点 当且仅当 对任意子集 S ⊆ L，其邻域 N(S) 满足：
- |N(S)| ≥ |S|
- 2) 最大匹配大小 = |L| - max_{(|S| - |N(S)|) | S ⊆ L}
- （若该值 < |L|，则无法完全覆盖 L）

4.4. 二分图最大匹配（匈牙利算法，稠密图且点数大于 1e4 不适用）

```
const int MAXN = 505; // 左侧集合点的最大数
量
vector<int> adj[MAXN]; // 邻接表，只存从左
往右的边
int match[MAXN];     // match[y] = x 表示右侧
点 y 匹配了左侧点 x
bool vis[MAXN];      // 标记右侧点在单次 DFS
中是否被访问过
bool dfs(int u) {
    for (int v : adj[u]) {
        if (!vis[v]) {
            vis[v] = true;
            // 如果右侧点没有匹配，或者原匹配点
可以找到新的增广路
            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}
int solve(int n_left) {
    int ans = 0;
    memset(match, -1, sizeof(match));
    for (int i = 1; i<= n_left; i++) {
        memset(vis, false, sizeof(vis)); // 每次都要重
```

```
置
        if (dfs(i)) ans++;
    }
    return ans;
}
```

4.5. 最小费用最大流（spfa）

```
struct MCMF {
    struct Edge { int to, rev; long long cap, cost; };
    vector<vector<Edge>> g;
    vector<long long> dist;
    vector<int> pre, pre_id;
    vector<bool> inq;
    void init(int n) { g.assign(n, {}); }
    void add_edge(int u, int v, long long cap, long
long cost) {
        g[u].push_back({v, (int)g[v].size(), cap,
cost});
        g[v].push_back({u, (int)g[u].size() - 1, 0, -
cost});
    }
bool spfa(int s, int t) {
        dist.assign(g.size(), 1e18);
        inq.assign(g.size(), false);
        pre.assign(g.size(), -1);
        pre_id.assign(g.size(), -1);
        queue<int> q;
        dist[s] = 0; q.push(s); inq[s] = true;
        while (!q.empty()) {
            int u = q.front(); q.pop(); inq[u] = false;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto &e = g[u][i];
                if (e.cap > 0 && dist[e.to] > dist[u] +
e.cost) {
                    dist[e.to] = dist[u] + e.cost;
                    pre[e.to] = u;
                    pre_id[e.to] = i;
                    if (!inq[e.to]) {
                        q.push(e.to);
                        inq[e.to] = true;
                    }
                }
            }
        }
        return dist[t] != 1e18;
    }
pair<long long, long long> min_cost_flow(int s,
int t) {
        long long flow = 0, cost = 0;
        while (spfa(s, t)) {
            long long f = 1e18;
            for (int v = t; v != s; v = pre[v])
                f = min(f, g[pre[v]][pre_id[v]].cap);
            flow += f;
            for (int v = t; v != s; v = pre[v]) {
                auto &e = g[pre[v]][pre_id[v]];
                e.cap -= f;
                g[v][e.rev].cap += f;
                cost += f * e.cost;
            }
        }
        return {flow, cost};
    }
};
如果费用非负且追求稳定复杂度，可用
Dijkstra + 势能 代替 SPFA。
```

4.6. 最大流（dinic 算法）

```
struct Dinic {
    struct Edge { int to, rev; long long cap; };
    vector<vector<Edge>> g;
    vector<int> lev, iter;
    void init(int n) { g.assign(n, {}); }
    void add_edge(int u, int v, long long cap) {
        g[u].push_back({v, (int)g[v].size(), cap});
        g[v].push_back({u, (int)g[u].size() - 1, 0});
    }
    void bfs(int s) {
        fill(lev.begin(), lev.end(), -1);
        queue<int> q;
        lev[s] = 0; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : g[u]) {
                if (e.cap > 0 && lev[e.to] == -1) {
                    lev[e.to] = lev[u] + 1;
                    q.push(e.to);
                }
            }
        }
    }
long long dfs(int u, int t, long long f) {
    if (u == t) return f;
    for (int &i = iter[u]; i < (int)g[u].size(); ++i) {
        auto &e = g[u][i];
        if (e.cap > 0 && lev[e.to] == lev[u] + 1) {
            long long d = dfs(e.to, t, min(f, e.cap));
            if (d > 0) {
                e.cap -= d;
                g[e.to][e.rev].cap += d;
                return d;
            }
        }
    }
    return 0;
}
long long max_flow(int s, int t) {
        long long flow = 0;
        lev.resize(g.size()); iter.resize(g.size());
        while (true) {
```

```
            bfs(s);
            if (lev[t] == -1) break;
            fill(iter.begin(), iter.end(), 0);
            long long f;
            while ((f = dfs(s, t, 1e18)) > 0) flow += f;
        }
        return flow;
    }
};
```

最小割：最大流 = 最小割。求完最大流后，从源点沿剩余容量 >0 的边 DFS，能到达的点集就是 S，其余为 T。

4.7. 有源汇最大流

```
struct BoundFlow {
    Dinic dinic;
    vector<long long> in;
    int S, T; // 超级源汇
    void init(int n) {
        dinic.init(n + 2);
        in.assign(n + 2, 0);
        S = n; T = n + 1;
    }
    // 添加有上下界的边 (u->v, l, r)
    void add_edge(int u, int v, long long l, long long r) {
        in[u] -= l; in[v] += l;
        dinic.add_edge(u, v, r - l);
    }
    bool feasible() {
        long long total = 0;
        for (int i = 0; i < S; ++i) { // S 是原图中最后一个点下标+1 ?
            if (in[i] > 0) {
                dinic.add_edge(S, i, in[i]);
                total += in[i];
            } else if (in[i] < 0) {
                dinic.add_edge(i, T, -in[i]);
            }
        }
        return dinic.max_flow(S, T) == total;
    }
// 有源汇 s,t 的最大流（先调用 add_edge 建图再调这个）
    long long max_flow_with_bound(int s, int t) {
        add_edge(t, s, 0, 1e18); // 加下限0的边
        if (!feasible()) return -1; // 无解
        long long flow = dinic.g[s].back().cap; // t->s 反向边容量 = 可行流中 s->t 的流量
        // 删掉 t->s 附加边
        dinic.g[t].pop_back(); dinic.g[s].pop_back();
        return flow + dinic.max_flow(s, t);
    }
};
```

上面 feasible() 中的 S 是 S 的点编号，初始化时 S = n，但循环里 i < S 跑到了 n-1，即所有原图点（0~n-1）。初始化时传入点数即可。
无源汇可行流
每个点计算 in[v] = 所有入边下限和 - 出边下限和。
建超级源 S 和超级汇 T。
若 in[v] > 0，连 S -> v 容量 in[v]；若 in[v] < 0，连 v -> T 容量 -in[v]。
原边连容量 r - l。
跑最大流，若从 S 出发的所有边满流，则有解

4.8. 费用流

```
using i64 = long long;
const i64 inf = 1e18;
struct edge{
    int t;
    int c;
    int v;
};
vector<edge> e;
vector<vector<int>> graph;
void add(int s, int t, int val) {
    graph[s].emplace_back(e.size());
    e.push_back({t, 1, val});
    graph[t].emplace_back(e.size());
    e.push_back({s, 0, -val});
    return ;
}
vector<i64> d;
vector<int> p, inq;

bool spfa(int s, int t) {
    int n = graph.size();

    d.assign(n, inf);
    p.assign(n, -1);
    inq.assign(n, 0);

    queue<int> q;
    d[s] = 0;
    q.push(s);
    inq[s] = 1;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = 0;

        for (int id : graph[u]) {
            auto &[to, cap, val] = e[id];

            if (cap && d[to] > d[u] + val) {
                d[to] = d[u] + val;
```

```
            p[to] = id;

            if (!inq[to]) {
              inq[to] = 1;
              q.push(to);
            }
          }
        }
    }

    return d[t] != inf;
}

pair<int, i64> mcf(int s, int t, int lim) {
    int flow = 0;
    i64 cost = 0;

    while (flow < lim && spfa(s, t)) {
        int f = lim - flow;

        // e[p[u] ^ 1].t 是 u 的前驱节点
        for (int u = t; u != s; u = e[p[u] ^ 1].t) {
            f = min(f, e[p[u]].c);
        }

        for (int u = t; u != s; u = e[p[u] ^ 1].t) {
            int id = p[u];

            e[id].c -= f;
            e[id ^ 1].c += f;
        }

        flow += f;
        cost += (i64)f * d[t];
    }

    return {flow, cost};
}
```

4.9. 点分治

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

// 点分治示例：统计距离 <= K 的无序点对 (u, v)，u != v。
// 单组输入：n K，随后 n-1 行 u v w。
// 1 <= n <= 200000，0 <= w <= 1e9，0 <= K <= 1e18。
// 时间 O(n log^2 n)，空间 O(n)。配套讲解见同目录的 讲解.md。
class CentroidDecomposition {
    using ll = long long;

    struct Edge {
        int to;
        ll weight;
    };

    struct State {
        int node, parent;
        ll distance;
    };

    vector<vector<Edge>> graph;
    vector<int> subtreeSize, parent;
    vector<bool> removed;
    ll limit;
    ll answer = 0;

    int findCentroid(int entry) {
        // order 保证父亲先于孩子出现；倒序即可自底向上算子树大小。
        // 使用迭代遍历，避免原树是一条长链时 DFS 爆栈。
        vector<int> order{entry};
        parent[entry] = 0;
        for (int i = 0; i < static_cast<int>(order.size()); ++i) {
            int u = order[i];
            for (const auto& e : graph[u]) {
                int v = e.to;
                if (removed[v] || v == parent[u]) continue;
                parent[v] = u;
                order.push_back(v);
            }
        }

        int total = static_cast<int>(order.size());
        int centroid = entry;
        int best = total;
        for (int i = total - 1; i >= 0; --i) {
            int u = order[i];
            subtreeSize[u] = 1;
            int largest = 0;
            for (const auto& e : graph[u]) {
                int v = e.to;
                if (removed[v] || v == parent[u]) continue;
                subtreeSize[u] += subtreeSize[v];
                largest = max(largest, subtreeSize[v]);
            }
            // 删除 u 后，父亲方向的连通块也必须考虑。
            largest = max(largest, total - subtreeSize[u]);
            if (largest < best) {
                best = largest;
```

```
                centroid = u;
            }
        }
        return centroid;
    }

    vector<ll> collectDistances(int start, int from,
ll firstDistance) const {
        vector<ll> distances;
        vector<State> stack{{start, from,
firstDistance}};
        while (!stack.empty()) {
            State current = stack.back();
            stack.pop_back();
            distances.push_back(current.distance);
            for (const auto& e : graph[current.node])
{
                if (removed[e.to] || e.to ==
current.parent) continue;
                stack.push_back({e.to, current.node,
current.distance + e.weight});
            }
        }
        return distances;
    }

    // F(a)：列表中下标 i < j 且 a[i] + a[j] <= K
的数对数。
    // 相同距离仍代表不同节点，不能去重。此
函数会原地排序。
    ll countPairs(vector<ll>& distances) const {
        sort(distances.begin(), distances.end());
        int left = 0;
        int right = static_cast<int>(distances.size()) -
1;
        ll result = 0;
        while (left < right) {
            if (distances[left] + distances[right] <=
limit) {
                result += right - left;
                ++left;
            } else {
                --right;
            }
        }
        return result;
    }

    ll countThrough(int centroid) const {
        vector<ll> all{0}; // 0 代表重心，统计以重
心为端点的路径。
        ll result = 0;
        for (const auto& e : graph[centroid]) {
            if (removed[e.to]) continue;
            vector<ll> distances =
collectDistances(e.to, centroid, e.weight);
            // 减去同分支：仍按“到当前重心的距
离和”计算。
            result -= countPairs(distances);
            all.insert(all.end(), distances.begin(),
distances.end());
        }
        result += countPairs(all);
        return result;
    }

    void decompose(int entry) {
        int centroid = findCentroid(entry);
        removed[centroid] = true;
        answer += countThrough(centroid);
        // countThrough 已返回，临时距离列表不
会留在递归栈中。
        for (const auto& e : graph[centroid]) {
            if (!removed[e.to]) decompose(e.to);
        }
    }

public:
    CentroidDecomposition(int n, ll k)
        : graph(n + 1), subtreeSize(n + 1), parent(n +
1),
          removed(n + 1, false), limit(k) {}

    void addEdge(int u, int v, ll weight) {
        graph[u].push_back({v, weight});
        graph[v].push_back({u, weight});
    }

    // 建完树后调用；重复调用时重置分治状
态。
    ll solve() {
        fill(removed.begin(), removed.end(), false);
        answer = 0;
        decompose(1);
        return answer;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;
    CentroidDecomposition solver(n, k);
    for (int i = 1; i < n; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.addEdge(u, v, w);
    }
```

```
    cout << solver.solve() << '\n';
    return 0;
}
```

5. 数据结构

5.1. 线段树

```
struct SegTree {
    using ll = long long;
    int n;
    vector<ll> sum, tag;

    // a[1..n] 为初始数组，a[0] 不用
    SegTree(const vector<ll>& a) {
        n = (int)a.size() - 1;
        sum.assign(4 * n + 4, 0);
        tag.assign(4 * n + 4, 0);
        build(1, 1, n, a);
    }

    void pull(int p) {
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }

    void build(int p, int l, int r, const vector<ll>&
a) {
        if (l == r) {
            sum[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }

    // 当前节点对应的区间整体加 v
    void apply(int p, int l, int r, ll v) {
        sum[p] += v * (r - l + 1);
        tag[p] += v;
    }

    void push(int p, int l, int r) {
        if (tag[p] == 0) return;
        int m = (l + r) / 2;
        apply(p * 2, l, m, tag[p]);
        apply(p * 2 + 1, m + 1, r, tag[p]);
        tag[p] = 0;
    }

    void add(int p, int l, int r, int L, int R, ll v) {
        if (L <= l && r <= R) {
            apply(p, l, r, v);
            return;
        }
        push(p, l, r);
        int m = (l + r) / 2;
        if (L <= m) add(p * 2, l, m, L, R, v);
        if (R > m) add(p * 2 + 1, m + 1, r, L, R, v);
        pull(p);
    }

    ll query(int p, int l, int r, int L, int R) {
        if (L <= l && r <= R) return sum[p];
        push(p, l, r);
        int m = (l + r) / 2;
        ll res = 0;
        if (L <= m) res += query(p * 2, l, m, L, R);
        if (R > m) res += query(p * 2 + 1, m + 1, r, L,
R);
        return res;
    }

    // 对外接口
    void add(int l, int r, ll v) {
        add(1, 1, n, l, r, v);
    }

    ll query(int l, int r) {
        return query(1, 1, n, l, r);
    }
};
```

5.2. 树状数组

```
struct BIT {
    using ll = long long;
    int n;
    vector<ll> tree;

    BIT(int n) : n(n), tree(n + 1, 0) {}

    // a[x] += v，注意 x 不能为 0
    void add(int x, ll v) {
        for (; x <= n; x += x & -x)
            tree[x] += v;
    }

    // 前缀和 a[1] + ... + a[x]，sum(0) = 0
    ll sum(int x) {
        ll res = 0;
        for (; x > 0; x -= x & -x)
            res += tree[x];
        return res;
    }

    ll query(int l, int r) {
        return sum(r) - sum(l - 1);
```

```
    }
};
```

5.3. bitset

```
const int MAXV = 100000; // 最大总和

bitset<MAXV + 1> dp;

int main() {
    dp[0] = 1; // 和为 0 可以达到

    vector<int> a = {3, 5, 7};

    for (int x : a) {
        dp |= dp << x;
    }

    cout << dp[10] << '\n'; // 是否能凑出 10
    cout << dp.count() << '\n'; // 一共能凑出多少种不同的和

    return 0;
}
```

```
#include <bits/stdc++.h>
using namespace std;

constexpr int N = 100000;
// 设 B = ceil(N / 64)，以下为常见 64 位实现的复杂度

int main() {
    bitset<N> b, other; // 初始化为全 0，O(B)
    int i = 3, k = 2;

    bool x = b[i];    // 读取第 i 位，O(1)
    b[i] = 1;         // 修改第 i 位，O(1)
    b.test(i);        // 读取第 i 位，带越界检查，O(1)

    b.set(i);         // 第 i 位设为 1，O(1)
    b.reset(i);       // 第 i 位设为 0，O(1)
    b.flip(i);        // 第 i 位取反，O(1)

    b.set();          // 全部设为 1，O(B)
    b.reset();        // 全部设为 0，O(B)
    b.flip();         // 全部取反，O(B)

    b.count();        // 1 的个数，O(B)
    b.any();          // 是否至少有一个 1，最坏 O(B)
    b.none();         // 是否全为 0，最坏 O(B)
    b.all();          // 是否全为 1，最坏 O(B)

    auto c = b << k;  // 左移 k 位，低位补 0，不修改 b，O(B)
    c = b >> k;       // 右移 k 位，高位补 0，不修改 b，O(B)
    b <<= k;          // 原地左移 k 位，O(B)
    b >>= k;          // 原地右移 k 位，O(B)

    c = b & other;    // 按位与，O(B)
    c = b | other;    // 按位或，O(B)
    c = b ^ other;    // 按位异或，O(B)
    c = ~b;           // 按位取反，不修改 b，O(B)

    b &= other;       // 原地按位与，O(B)
    b |= other;       // 原地按位或，O(B)
    b ^= other;       // 原地按位异或，O(B)

    bool eq = (b == other); // 是否相等，最坏 O(B)
    b = other;        // 复制另一个 bitset，O(B)

    string s = b.to_string(); // 转成字符串，高位在左，O(N)

    return 0;
}

// 注意：
// 1. N 必须是编译期常量，下标范围为 [0, N - 1]。
// 2. 移位丢弃超出范围的位；k >= N 时结果全为 0。
// 3. 空间约 N / 8 字节，实际按机器字对齐。
// 4. 上述复杂度是常见实现的估算，并非标准统一保证。
```

5.4. 可删除并查集

```
#include <iostream>
#include <numeric>
#include <vector>
using namespace std;

class DeletableDSU {
private:
    vector<int> parent;
    vector<int> setSize;
    vector<int> id;
    vector<long long> setSum;
    int nodeCount;

public:
    DeletableDSU(int n, int maxOperations)
        : parent(n + maxOperations + 1),
          setSize(n + maxOperations + 1),
```

```
        id(n + 1),
        setSum(n + maxOperations + 1),
        nodeCount(n) {
    for (int i = 1; i <= n; ++i) {
        parent[i] = i;
        setSize[i] = 1;
        setSum[i] = i;
        id[i] = i;
    }
}

int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]);
    }
    return parent[x];
}

void unite(int x, int y) {
    int rootX = find(id[x]);
    int rootY = find(id[y]);

    if (rootX == rootY) {
        return;
    }

    if (setSize[rootX] > setSize[rootY]) {
        swap(rootX, rootY);
    }

    parent[rootX] = rootY;
    setSize[rootY] += setSize[rootX];
    setSum[rootY] += setSum[rootX];
}

// 将 x 从原集合删除，并让它成为单元素集合
void erase(int x) {
    int oldRoot = find(id[x]);

    --setSize[oldRoot];
    setSum[oldRoot] -= x;

    ++nodeCount;
    id[x] = nodeCount;
    parent[nodeCount] = nodeCount;
    setSize[nodeCount] = 1;
    setSum[nodeCount] = x;
}

// 将 x 移动到 y 所在的集合
void move(int x, int y) {
    int rootX = find(id[x]);
    int rootY = find(id[y]);

    if (rootX == rootY) {
        return;
    }

    --setSize[rootX];
    setSum[rootX] -= x;

    ++nodeCount;
    id[x] = nodeCount;
    parent[nodeCount] = rootY;

    ++setSize[rootY];
    setSum[rootY] += x;
}

pair<int, long long> query(int x) {
    int root = find(id[x]);
    return {setSize[root], setSum[root]};
}
};
```

5.5. 关于 dsu 的一些碎碎念

```
//按秩合并还是有点必要的,小的合并到大的才能保证正确的复杂度
struct dsu{
    vector<int> fa;
    vector<int> siz;
    dsu(int n) : fa(n + 1), siz(n + 1, 1) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) {
        if (x != fa[x]) return fa[x] = find(fa[x]);
        return fa[x];
    }
    bool unit(int x, int y) {
        int fx = find(x), fy = find(y);
        if (fx == fy) return 0;
        if (siz[fx] > siz[fy]) swap(fx, fy);
        fa[fx] = fy;
        siz[fy] += siz[fx];
        return 1;
    }
};
```

5.6. 带权并查集

```
/*
题目如下:
有 n 个电脑配件，质量分别为 x_1,x_2,...,x_n。
x_i+x_j=2c.
对于每次测量结果，你需要判断它此时是否有可能是正确的。
也就是说，如果存在一组实数 x_1,x_2,...,x_n 同时满足当前测量结果和之前被认为正确的全部测量结果，则认为该结果正确。被认为错误
```

```
的测量结果不会对之后的询问产生任何影响。
注意，因为电脑配件可能由奇异物质组成，所以 x_i 可能是负的。
i,j,c = a_i + a_j = 2 * c
*/
struct dsu {
    vector<i64> fa, sgn, d, siz, fixed, val;
    dsu(int n) : fa(n + 1), sgn(n + 1, 1), d(n + 1),
siz(n + 1, 1), fixed(n + 1, 0), val(n + 1) {
        iota(fa.begin(), fa.end(), 0);
    }//sgn,fixed,d和val是题目维护的量
    int find(int x) {
        if (x == fa[x]) return x;
        int oldfa = fa[x], newfa = find(oldfa);
        if (oldfa == newfa) return oldfa;
        d[x] += sgn[x] * d[oldfa];
        sgn[x] *= sgn[oldfa];
        fa[x] = newfa;
        return newfa;
    }
    bool solve(int x, int y, int c) {
        int fax = find(x), fay = find(y);
        if (fax == fay) {
            int newsgn = sgn[x] + sgn[y];
            if (newsgn == 0)  return d[x] + d[y] == 2 *
c;
            i64 newval = (2 * c - d[x] - d[y]) / newsgn;
            if (!fixed[fax]) {
                fixed[fax] = 1;
                val[fax] = newval;
                return 1;
            }
            return val[fax] == newval;
        }
        if (siz[fax] > siz[fay]) {
            swap(fax, fay);
            swap(x, y);
        }
        if (fixed[fax] && fixed[fay])
            return sgn[x] * val[fax] +
            sgn[y] * val[fay] +
            d[x] + d[y] == 2 * c;
        fa[fax] = fay;
        siz[fay] += siz[fax];
        int oldsgn = sgn[x];
        sgn[fax] = -oldsgn * sgn[y];
        d[fax] = oldsgn * (2 * c - d[x] - d[y]);
        if (fixed[fax] && !fixed[fay]) {
            fixed[fay] = 1;
            val[fay] = sgn[fax] * (val[fax] - d[fax]);
        }
        return 1;
    }
};
```

5.7. AC 自动机

```
#include <array>
#include <iostream>
#include <queue>
#include <string>
#include <vector>
using namespace std;

struct AhoCorasick {
    static constexpr int SIGMA = 26;
    struct Node {
        array<int, SIGMA> go{};
        int fail = 0;
        long long visit = 0;
    };

    vector<Node> tr;
    AhoCorasick() { tr.emplace_back(); } // 0 为根

    int insert(const string &s) {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!tr[u].go[c]) {
                tr[u].go[c] = (int)tr.size();
                tr.emplace_back();
            }
            u = tr[u].go[c];
        }
        return u;
    }

    vector<int> build() {
        queue<int> q;
        vector<int> order;
        order.reserve(tr.size());
        for (int c = 0; c < SIGMA; ++c) {
            int v = tr[0].go[c];
            if (v) q.push(v);
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            order.push_back(u);
            for (int c = 0; c < SIGMA; ++c) {
                int &v = tr[u].go[c];
                if (v) {
                    tr[v].fail = tr[tr[u].fail].go[c];
                    q.push(v);
                } else {
                    v = tr[tr[u].fail].go[c];
                }
            }
        }
        return order;
    }
```

```
    void match(const string &text) {
        int u = 0;
        for (char ch : text) {
            u = tr[u].go[ch - 'a'];
            ++tr[u].visit;
        }
    }

    void collect(const vector<int> &order) {
        for (int i = (int)order.size() - 1; i >= 0; --i) {
            int u = order[i];
            tr[tr[u].fail].visit += tr[u].visit;
        }
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    AhoCorasick ac;
    vector<int> terminal(n);
    string pattern;
    for (int i = 0; i < n; ++i) {
        cin >> pattern;
        terminal[i] = ac.insert(pattern);
    }
    string text;
    cin >> text;

    vector<int> order = ac.build();
    ac.match(text);
    ac.collect(order);

    for (int node : terminal) cout <<
ac.tr[node].visit << '\n';
    return 0;
}
```

5.8. 笛卡尔树(区间最值,区间贡献)

```
//代码为小根笛卡尔树
#include <bits/stdc++.h>
using namespace std;

struct CartesianTree {
    int n, root;
    vector<int> lc, rc, parent;

    // a[1..n] 为原数组；即使 n=0，也要保留
a[0]。
    explicit CartesianTree(const vector<long
long>& a)
        : n((int)a.size() - 1), root(0),
          lc(n + 1, 0), rc(n + 1, 0), parent(n + 1, 0) {
        vector<int> st;
        st.reserve(n);

        for (int i = 1; i <= n; ++i) {
            int last = 0;
            while (!st.empty() && a[st.back()] > a[i]) {
                last = st.back();
                st.pop_back();
            }

            if (!st.empty()) {
                rc[st.back()] = i;
                parent[i] = st.back();
            }
            if (last != 0) {
                lc[i] = last;
                parent[last] = i;
            }
            st.push_back(i);
        }

        if (!st.empty()) root = st.front();
    }

    // 返回一种孩子先于父亲出现的处理顺序，
方便做树形 DP。
    vector<int> bottom_up_order() const {
        vector<int> order;
        order.reserve(n);
        if (root == 0) return order;

        vector<int> st{root};
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            order.push_back(u);
            if (lc[u]) st.push_back(lc[u]);
            if (rc[u]) st.push_back(rc[u]);
        }
        reverse(order.begin(), order.end());
        return order;
    }
};
```

5.9. 虚树

```
#include <bits/stdc++.h>
using namespace std;

// 虚树下标为 0..m-1，nodes[i] 才是原树节点
编号。
// nodes 按原树 DFS 序排列，根在下标 0，父
```

```
亲下标始终小于孩子下标。
struct VirtualTree {
    vector<int> nodes;
    vector<int> parent;      // 虚树父亲下标，根为 -1
    vector<int> length;      // 父亲到当前节点的原树距离，根为 0
    vector<char> is_key;     // 是否属于本次询问的关键点集合
};

class VirtualTreeBuilder {
    int n, LOG;
    vector<vector<int>> graph, up;
    vector<int> depth, tin, tout;

public:
    explicit VirtualTreeBuilder(int size) : n(size), LOG(1), graph(n + 1),
        depth(n + 1), tin(n + 1), tout(n + 1) {
        while ((1LL << LOG) <= n) ++LOG;
        up.assign(LOG, vector<int>(n + 1));
    }

    void add_edge(int u, int v) {
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    // 原树是连通无向树；固定根后预处理一次。
    // 显式栈模拟 DFS，避免链形树的递归栈溢出。
    void preprocess(int root = 1) {
        vector<int> next_edge(n + 1), st{root};
        int timer = 0;
        depth[root] = 0;
        up[0][root] = root;
        while (!st.empty()) {
            int u = st.back();
            if (next_edge[u] == 0) {
                tin[u] = ++timer;
                for (int j = 1; j < LOG; ++j)
                    up[j][u] = up[j - 1][up[j - 1][u]];
            }
            if (next_edge[u] == (int)graph[u].size()) {
                // 子树的 DFS 序区间为 [tin[u], tout[u]]。
                tout[u] = timer;
                st.pop_back();
                continue;
            }
            int v = graph[u][next_edge[u]++];
            if (v == up[0][u]) continue;
            up[0][v] = u;
            depth[v] = depth[u] + 1;
            st.push_back(v);
        }
    }

    bool is_ancestor(int u, int v) const {
        return tin[u] <= tin[v] && tin[v] <= tout[u];
    }

    int lca(int u, int v) const {
        if (is_ancestor(u, v)) return u;
        if (is_ancestor(v, u)) return v;
        for (int j = LOG - 1; j >= 0; --j)
            if (!is_ancestor(up[j][u], v)) u = up[j][u];
        return up[0][u];
    }

    VirtualTree build(vector<int> keys) const {
        VirtualTree result;
        if (keys.empty()) return result;
        auto by_dfs = [&](int a, int b) { return tin[a] < tin[b]; };
        sort(keys.begin(), keys.end(), by_dfs);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());

        result.nodes = keys;
        int k = (int)keys.size();
        // 只对去重后的原始关键点取相邻 LCA，不遍历正在扩大的 nodes。
        for (int i = 1; i < k; ++i)
            result.nodes.push_back(lca(keys[i - 1], keys[i]));
        sort(result.nodes.begin(), result.nodes.end(), by_dfs);
        result.nodes.erase(unique(result.nodes.begin(), result.nodes.end()),
                           result.nodes.end());

        int m = (int)result.nodes.size();
        result.parent.assign(m, -1);
        result.length.assign(m, 0);
        result.is_key.assign(m, false);
        vector<int> st; // 存虚树下标，保持一条祖先链
        st.reserve(m);
        int key_pos = 0;
        for (int i = 0; i < m; ++i) {
            int u = result.nodes[i];
            // 两个数组都按 DFS 序排列，可以线性标记关键点。
            if (key_pos < k && u == keys[key_pos]) {
                result.is_key[i] = true;
                ++key_pos;
            }
```

```
        while (!st.empty() && !
is_ancestor(result.nodes[st.back()], u))
            st.pop_back();
        if (!st.empty()) {
            int p = st.back();
            result.parent[i] = p;
            result.length[i] = depth[u] -
depth[result.nodes[p]];
        }
        st.push_back(i);
    }
    return result;
  }
};

// 每一对不同关键点只计算一次；重复编号按集合去重。
long long pair_distance_sum(const VirtualTree&
tree) {
  int m = (int)tree.nodes.size();
  vector<long long> count(m);
  long long k = 0, answer = 0;
  for (int i = 0; i < m; ++i) {
    count[i] = tree.is_key[i];
    k += count[i];
  }
  // 父亲先于孩子出现，倒序就是合法的自底向上处理顺序。
  for (int i = m - 1; i >= 0; --i) {
    int p = tree.parent[i];
    if (p == -1) continue;
    answer += 1LL * tree.length[i] * count[i] * (k
- count[i]);
    count[p] += count[i];
  }
  return answer;
}

// 输入：n q，n-1 条无权边；每个询问为 k 和 k 个关键点编号。
// 输出：每个询问的关键点两两距离之和。
int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n, q;
  if (!(cin >> n >> q)) return 0;
  VirtualTreeBuilder builder(n);
  for (int i = 1; i < n; ++i) {
    int u, v;
    cin >> u >> v;
    builder.add_edge(u, v);
  }
  builder.preprocess(1);
  while (q--) {
    int k;
    cin >> k;
    vector<int> keys(k);
    for (int& u : keys) cin >> u;
    VirtualTree tree =
builder.build(move(keys));
    cout << pair_distance_sum(tree) << '\n';
  }
  return 0;
}
```

6. 马拉车

```
// d[i] 表示以 i 为中心的最长回文半径
// 转换后的字符串长度为 2n+1，d[i]-1 即为原字符串中对应回文串的长度
vector<int>manacher(string s) {
  string t = "#";
  for (char c : s) t += c, t += "#";
  int n = t.size();
  vector<int>d(n);
  // r 为当前已知回文串覆盖的最右边界，l 为该边界对应的中心
  for (int i = 0, l = 0, r = -1; i<n; i++) {
    int k = (i>r) ? 1 : min(d[l + r - i], r - i + 1);
    while (0<= i - k&&i + k<n&&t[i - k] == t[i +
k]) k++;
    d[i] = k--;
    if (i + k>r) l = i - k, r = i + k;
  }
  return d;
}
```

7. 拉格朗日插值（通用版，$o(n^2)$）

```
// n 个点 (x[i], y[i])，求 f(k)
ll lagrange(int n, ll k, vector<ll>&x,
vector<ll>&y) {
  ll ans = 0;
  for (int i = 0; i<n; i++) {
    ll up = y[i], down = 1;
    for (int j = 0; j<n; j++) {
      if (i == j) continue;
      up = up * (k - x[j] % MOD + MOD) %
MOD;
      down = down * (x[i] - x[j] % MOD +
MOD) % MOD;
    }
    ans = (ans + up * inv(down)) % MOD;
  }
  return ans;
}
```

8. 拉格朗日插值（连续整数）

```
ll pre[MAXN], suf[MAXN], fac[MAXN],
inv_fac[MAXN];
// x_i 为 1, 2, ..., n 的情况， 求 f(k)
ll lagrange_linear(int n, ll k, vector<ll>&y) {
    if (k >= 1 && k <= n) return y[k - 1];

    pre[0] = suf[n + 1] = 1;
    for (int i = 1; i<= n; i++) pre[i] = pre[i - 1] * (k -
i % MOD + MOD) % MOD;
    for (int i = n; i>= 1; i--) suf[i] = suf[i + 1] * (k -
i % MOD + MOD) % MOD;

    fac[0] = 1;
    for (int i = 1; i<= n; i++) fac[i] = fac[i - 1] * i %
MOD;
    inv_fac[n] = inv(fac[n]);
    for (int i = n - 1; i>= 0; i--) inv_fac[i] =
inv_fac[i + 1] * (i + 1) % MOD;

    ll ans = 0;
    for (int i = 1; i<= n; i++) {
        ll up = pre[i - 1] * suf[i + 1] % MOD * y[i - 1]
% MOD;
        ll down = inv_fac[i - 1] * inv_fac[n - i] %
MOD;
        // 分母符号取决于 (n-i) 的奇偶性
        if ((n - i) % 2) ans = (ans - up * down % MOD
+ MOD) % MOD;
        else ans = (ans + up * down % MOD) %
MOD;
    }
    return ans;
}
```

9. Odt

```
struct Node {
    int l, r;
    mutable ll v; // mutable 允许在 set 中直接修
改 v
    Node(int L, int R = -1, ll V = 0) : l(L), r(R), v(V)
{}
    bool operator<(const Node& t) const { return
l<t.l; }
};

set<Node>odt;

// 分裂操作：将包含 pos 的区间 [l, r] 分裂为 [l,
pos-1] 和 [pos, r]
// 返回后一个区间的迭代器
auto split(int pos) {
    auto it = odt.lower_bound(Node(pos));
    if (it != odt.end() && it->l == pos) return it;
    --it;
    int l = it->l, r = it->r;
    ll v = it->v;
    odt.erase(it);
    odt.insert(Node(l, pos - 1, v));
    return odt.insert(Node(pos, r, v)).first;
}

// 核心：区间赋值（推平操作）
void assign(int l, int r, ll v) {
    auto itR = split(r + 1), itL = split(l); // 顺序必须
先 R 后 L
    odt.erase(itL, itR);
    odt.insert(Node(l, r, v));
}

// 其他区间操作模板（以区间加为例）
void add(int l, int r, ll v) {
    auto itR = split(r + 1), itL = split(l);
    for (; itL != itR; ++itL) itL->v += v;
}
```

10. 前缀数组（来自 oiwiki）

```
vector<int> prefix_function(string s) {
  int n = (int)s.length();
  vector<int> pi(n);
  for (int i = 1; i < n; i++) {
    int j = pi[i - 1];
    while (j > 0 && s[i] != s[j]) j = pi[j - 1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
  }
  return pi;
}
```

11. kmp（来自 oiwiki）

```
vector<int>find_occurrences(string text, string
pattern) {
  string cur = pattern + '#' + text;
  int sz1 = text.size(), sz2 = pattern.size();
  vector<int>v;
  vector<int>lps = prefix_function(cur);
  for (int i = sz2 + 1; i<= sz1 + sz2; i++) {
    if (lps[i] == sz2) v.push_back(i - 2 * sz2);
  }
  return v;
}
```

12. mt19937

```
#include<bits/stdc++.h>
int main() {
```

```
    std::random_device rd;
    std::mt19937 gen(rd());

    std::uniform_int_distribution<>dis(1, 100);
    //
std::uniform_real_distribution<double>dis(0.0,
1.0);
    for (int i = 0; i<5; ++i) {
        std::cout << dis(gen) << " ";
    }

    return 0;
}
```

13. 线性基

```
typedef long long ll;
const int MAXL = 60;
struct LinearBasis {
    ll a[MAXL + 1];

    LinearBasis() {
        memset(a, 0, sizeof(a));
    }

    // 简洁插入（不消元，只保证线性无关）
    void insert(ll x) {
        for (int i = MAXL; i>= 0; --i) {
            if (!(x >> i & 1)) continue;
            if (!a[i]) { a[i] = x; return; }
            x ^= a[i];
        }
    }

    // 查询异或最大值（贪心即可，不需要消
元）
    ll queryMax() {
        ll res = 0;
        for (int i = MAXL; i>= 0; --i)
            if ((res ^ a[i])>res) res ^= a[i];
        return res;
    }

    // 合并另一个线性基
    void merge(const LinearBasis&other) {
        for (int i = 0; i<= MAXL; ++i)
            if (other.a[i]) insert(other.a[i]);
    }
};
int main() {
    int n;
    cin>>n;
    LinearBasis lb;
    for (int i = 0; i<n; ++i) {
        ll x;
        cin>>x;
        lb.insert(x);
    }
    cout << lb.queryMax() << "\n";
    return 0;}
```

13.1. 异或线性基典题

n个二元组<A,B>,求一个非空子集使得A的异或和不为0且B的和最大
solution:B从大到小排列,然后加入后如果异或为0则不取这组,根据(a^b=c && a^c=b)和贪心可证明正确性

```
/*
线性基+树上rmq
题目:速求树上任意两点间的路径上的点值构成
的子集的异或最大/最小值
*/
#include<iostream>
#include<vector>
using namespace std;
using i64 = long long;
struct linearbasis{
    vector<i64> basis;
    linearbasis(int n) : basis(n + 1){}
    void insert(i64 x) {
        for (int i = 63; i >= 0; i--) {
            if ((x >> i) & 1) {
                if (basis[i]) x ^= basis[i];
                else return basis[i] = x, void();
            }
        }
        return ;
    }
    linearbasis comb(linearbasis a, linearbasis b) {
        for (int i = 63; i >= 0; i--) {
            if (b.basis[i]) a.insert(b.basis[i]);
        }
        return a;
    }
};
void solve() {
    int n, q;
    cin >> n >> q;
    vector<i64> g(n + 1);
    int len = 0;
    while((1 << len) <= n) len++;
    for (int i = 1; i <= n; i++) cin >> g[i];
    vector<vector<int>> e(n + 1);
    for (int i = 0, u, v; i < n - 1; i++) {
        cin >> u >> v;
        e[u].push_back(v);
        e[v].push_back(u);
    }
```

```
    vector<vector<linearbasis>> rmq(len,
vector<linearbasis>(n + 1, linearbasis(64)));
    vector<vector<int>> lca(len, vector<int>(n +
1));
    vector<int> depth(n + 1, 1);
    auto dfs = [&](auto &&self, int p, int fa) -
>void{
        rmq[0][p].insert(g[p]);
        for (auto i : e[p]) {
            if (i == fa) continue;
            lca[0][i] = p;
            depth[i] = depth[p] + 1;
            self(self, i, p);
        }
    };
    dfs(dfs, 1, 0);
    for (int i = 1; i < len; i++) {
        for (int j = 1; j <= n; j++) {
            lca[i][j] = lca[i - 1][lca[i - 1][j]];
        }
    }
    for (int i = 1; i < len; i++) {
        for (int j = 1; j <= n; j++) {
            rmq[i][j] = rmq[i][j].comb(rmq[i - 1][j],
rmq[i - 1][lca[i - 1][j]]);
        }
    }
    auto cal_lca = [&](int x, int y){
        if (depth[x] < depth[y]) swap(x, y);
        int dis = depth[x] - depth[y];
        for (int i = len; i >= 0; i--) {
            if ((dis >> i) & 1) x = lca[i][x];
        }
        if (x == y) return x;
        for (int i = len - 1; i >= 0; i--) {
            if (lca[i][x] != lca[i][y]) {
                x = lca[i][x];
                y = lca[i][y];
            }
        }
        return lca[0][x];
    };
    for (int i = 0, x, y; i < q; i++) {
        cin >> x >> y;
        int _lca = cal_lca(x, y);
        //cout << _lca << '\n';
        linearbasis lb(64);
        lb.insert(g[_lca]);
        int dis = depth[x] - depth[_lca];
        for (int j = len; j >= 0; j--) {
            if ((dis >> j) & 1) {
                lb = lb.comb(lb, rmq[j][x]);
                x = lca[j][x];
            }
        }
        //cout << dis << ' ' << x << '\n';
        dis = depth[y] - depth[_lca];
        for (int j = len; j >= 0; j--) {
            if ((dis >> j) & 1) {
                lb = lb.comb(lb, rmq[j][y]);
                y = lca[j][y];
            }
        }
        //cout << dis << ' ' << x << '\n';
        i64 ans = 0;
        for (int j = 63; j >= 0; j--) {
            ans = max(ans, ans ^ lb.basis[j]);
        }
        cout << ans << '\n';
    }
    return ;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    solve();
    return 0;
}
```

14. 强连通分量-tarjan

```
/*
    关于tarjan算法的一些个人理解(待补充)
*/
const int N = 1e5 + 5;
vector<int>g[N];
int dfn[N], low[N], timer;
bool inStk[N];
vector<int> stk;                // 模拟栈
vector<vector<int>> scc;        // 收集所有强连
通分量

void tarjan(int u) {
    dfn[u] = low[u] = ++timer;
    stk.push_back(u);
    inStk[u] = true;
    for (int v : g[u]) {
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (inStk[v]) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] == low[u]) {         // u 是 SCC 的根
        vector<int>comp;
        while (true) {
            int v = stk.back();
            stk.pop_back();
            inStk[v] = false;
            comp.push_back(v);
```

```
        if (v == u) break;
    }
    scc.push_back(comp); }}      // 收集当前分量
int main() {
  read(n,m,g);//g为edge
  for (int i = 1; i<= n; ++i)
    if (!dfn[i]) tarjan(i);
  // 输出所有强连通分量
  for (int i = 0; i<(int)scc.size(); ++i) {
    cout << "SCC " << i + 1 << ": ";
    for (int x : scc[i]) cout << x << " ";
    cout << "\n";
  }
  return 0;}
```

15. polya 定理

$|\frac{C}{G}| = \frac{1}{|G|}\sum_{g\in G} m^{c(g)}$

- 置换群
- 群的阶（置换个数）
- 所有着色方案（共 m^n 种）
- 轨道集合（本质不同着色）
- 可用颜色数
- 置换 g 的循环个数
- 在 g 下保持不变的着色数

$|\frac{X}{G}| = \frac{1}{|G|}\sum_{g\in G}|X^g|$

补充：burnside 引理

- 被作用的集合
- 轨道集
- 群阶
- g 作用下的不动点集

16. 求割点

```
const int N = 1e5 + 5;
vector<int>G[N];
int dfn[N], low[N], timer;
bool cut[N];        // cut[u] = true 表示 u 是割点

void tarjan(int u, int fa) {
  dfn[u] = low[u] = ++timer;
  int child = 0;              // 子树个数（仅对根有用）
  for (int v : G[u]) {
    if (v == fa) continue;
    if (!dfn[v]) {
      child++;
      tarjan(v, u);
      low[u] = min(low[u], low[v]);
      if (low[v] >= dfn[u]) cut[u] = true;  // 割点判定
    } else {
      low[u] = min(low[u], dfn[v]);
    }
  }
  if (fa == -1) cut[u] = (child >= 2);  // 根节点特判
}
// 调用：
// for (int i = 1; i<= n; i++)
//   if (!dfn[i]) tarjan(i, -1);
```

17. 自定义 hash

```
struct Person {
  std::string name;
  int age;
};
// 必须在 std 中打开 namespace
namespace std {
  template<>
  struct hash<Person>{
    size_t operator()(const Person&p) const noexcept {
      // 组合多个成员的哈希值（常用方法见后文）
      size_t h1 = hash<string>{}(p.name);
      size_t h2 = hash<int>{}(p.age);
      return h1 ^ (h2 << 1);  // 简单组合，更稳健的方法见下
    }
  };
}
```

```
namespace std {
  template<typename T1, typename T2>
  struct hash<pair<T1, T2>>{
    size_t operator()(const pair<T1, T2>&p) const {
      auto h1 = hash<T1>{}(p.first);
      auto h2 = hash<T2>{}(p.second);
      // 组合方法...
    }
  };
}
```

```
struct Point {
  int x, y;
};
namespace std {
  template<>
  struct hash<Point>{
    size_t operator()(const Point&p) const {
      size_t seed = 0;
      hash<int>hasher;
      // 对每个字段依次调用 hash_combine
```

```
        hash_combine(seed, hasher(p.x));
        hash_combine(seed, hasher(p.y));
        return seed;
    }
  };
}
inline void hash_combine(size_t& seed, size_t hash) {
  seed ^= hash + 0x9e3779b97f4a7c15ULL + (seed << 6) + (seed >> 2);
}
/*防卡unordered_map教学https://codeforces.com/blog/entry/62393
unordered_map<long long, int, custom_hash> safe_map;
*/
```

18. Zobrist Hashing（xor hashing）

赋值：每个值 v → 随机 64 位整数 hash[v]

前缀：pre[i] = pre[i-1] ^ hash[A[i]]

比较：子数组 XOR = pre[r] ^ pre[l-1]，与预期哈希比较

多测放全局开 mt19937

```
//mt19937 rng32(...);  // 生成 uint32_t
//mt19937_64 rng64(...); // 生成 uint64_t
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
hash[i] = rng();
```

19. Lemma (Bertrand's postulate)

For each positive integer xx, there is a prime pp inside the interval [x,2x].

20. 半开半闭区间二分

l为左端点,r为右端点+1
结束时l == r, [L, r),返回l或r均可

21. 开区间二分(l,r)

它的核心思想可以总结为“红蓝染色法”（维护不变量）。
💡 核心法则：我们将数组分为两半，一半是不满足条件的（红色，设为 l），一半是满足条件的（蓝色，设为 r）。
l 始终指向“不满足条件”的位置。r 始终指向“满足条件”的位置。
1. 初始化：因为 l 必须一开始就不满足条件，r 必须一开始就满足条件，我们要把它们放在数组界外（或者已知的边界上）。对于一个长度为 m 的数组 b（下标 0 - m - 1）

```
int l = -1; // -1 绝对不满足条件（越界了，算作不满足）
int r = m;  // m 绝对满足条件（我们要找的答案一定在 m 左侧，所以 m 兜底）
```

2. 循环条件：既然是开区间 (l, r)，说明 l 和 r 之间还有元素。只要它们俩之间还有至少一个元素，就继续二分

```
while (l + 1 < r) // 当 l 和 r 相邻时（比如 l=2, r=3），循环结束
```

3. 状态转移（极度舒适，不需要 +1 或 -1）

```
int mid = l + (r - l) / 2;
if (check(mid) == true) {
    r = mid; // mid 满足条件，所以把"蓝边界" r 移到 mid
} else {
    l = mid; // mid 不满足条件，所以把"红边界" l 移到 mid
}
```

4. 退出循环时的特点（重点！）：退出时，必定有 l + 1 == r（即 l 和 r 紧紧挨在一起）。因为 l 始终是不满足条件的最后一个位置，r 始终是满足条件的第一个位置。所以，你要找的目标答案就是 r！ 根本不用额外去推导什么边界。

22. 数位 dp

```
// 递归版,看个大概,具体题目具体分析，下面这个是给定上限，求满足a,b,c a^b^c == 0 && a+c == 2*b的a,b,c个数
vector<i64> pre(64);
i64 memory[64][2][3][2];
i64 n;

i64 pre_dp(int bit, int choice, int limit, int tight, i64 upper_n) {
    int n_bit = (upper_n >> bit) & 1;

    if (tight && choice > n_bit) return 0;

    auto &p = memory[bit][choice][limit][tight];
    if (p != -1) return p;

    if (bit == 0) {
        return p = (limit != 1);
    }

    int next_tight = tight && (choice == n_bit);
    if (limit == 1) {
        return p = pre_dp(
            bit - 1, 1, 2, next_tight, upper_n
        );
```

```
    }

    if (limit == 2) {
        p = pre_dp(
                bit - 1, 1, 1, next_tight, upper_n
            )
          + pre_dp(
                bit - 1, 0, 0, next_tight, upper_n
            );

        return p %= MOD;
    }
    p = pre_dp(
            bit - 1, 1, 0, next_tight, upper_n
        )
      + pre_dp(
            bit - 1, 0, 0, next_tight, upper_n
        );

    return p %= MOD;
}
//递推版,上下不是同一题
string S;  // 上限
int n = S.size();
// dp[pos][state][tight]
// state 按题目定义（余数/和/积等）
vector<vector<vector<int>>> dp(n + 1,
vector<vector<int>>(MAX_STATE,
vector<int>(2, 0)));
dp[0][初始状态][1] = 1;  // 第0位，初始状态，受限
for (int i = 0; i < n; i++) {
    for (int st = 0; st < MAX_STATE; st++) {
        for (int tight = 0; tight < 2; tight++) {
            if (dp[i][st][tight] == 0) continue;

            int limit = tight ? (S[i] - '0') : 9;
            for (int d = 0; d <= limit; d++) {
                int nst = 转移(st, d);          // 按题目改
                int ntight = tight && (d == limit);
                dp[i + 1][nst][ntight] = (dp[i + 1][nst]
[ntight] + dp[i][st][tight]) % MOD;
            }
        }
    }
}
// 最终答案：所有 tight 状态的和
for (int tight = 0; tight < 2; tight++)
    ans = (ans + dp[n][目标状态][tight]) % MOD;
string S;  // 上限数字的字符串
int D;     // 题目参数
int n;
int memo[10005][105][2][2];  // 按题目需求调整大小
int dfs(int pos, int sum, bool tight, bool leadZero)
{
    if (pos == n) {
        return sum == 0 ? 1 : 0;  // 终止条件按题目改
    }
    if (memo[pos][sum][tight][leadZero] != -1)
return memo[pos][sum][tight][leadZero];
    int res = 0;
    int limit = tight ? (S[pos] - '0') : 9;
    for (int d = 0; d <= limit; d++) {
        bool ntight = tight && (d == limit);
        bool nleadZero = leadZero && (d == 0);

        if (leadZero && d == 0) {
            res = (res + dfs(pos + 1, sum, ntight, true))
% MOD;  // 前导零，不贡献
        } else {
            int nsum = (sum + d) % D;  // 状态更新按题目改
            res = (res + dfs(pos + 1, nsum, ntight,
false)) % MOD;
        }
    }
    return memo[pos][sum][tight][leadZero] =
res;
}
```

23. 关于 gcd 的常数优化

```
int gcd(int a, int b) {
    int az = __builtin_ctz(a);
    int bz = __builtin_ctz(b);
    int z = min(az, bz);
    b >>= bz;
    while (a) {
        a >>= az;
        int diff = a - b;
        az = __builtin_ctz(diff);
        b = min(a, b), a = abs(diff);
    }
    return b << z;
}
```

```
vector<array<int,3>> cal_(int x) {
    vector<array<int,3>> re(x + 1, {1, 1, 1});
    vector<int> not_prime(x + 1, 0);
    vector<int> prime;
    prime.reserve(x / 10);
    for (int i = 2; i <= x; i++) {
        if (!not_prime[i]) {
            prime.emplace_back(i);
            re[i] = {1, 1, i};
        }
        for (int j : prime) {
```

```
            if (i * j > x) break;
            not_prime[i * j] = 1;
            auto &[a, b, c] = re[i];
            int nj = a * j;
            if (nj <= b) {
                re[i * j] = {nj, b, c};
            } else if (nj <= c) {
                re[i * j] = {b, nj, c};
            } else {
                re[i * j] = {b, c, nj};
            }

            if (i % j == 0) break;
        }
    }
    return re;
}
int fast_gcd(int a, int b) {
    if (!a || !b) return (a | b);
    int az = __builtin_ctz(a);
    int bz = __builtin_ctz(b);
    int z = min(az, bz);
    b >>= bz;
    while(a) {
        a >>= az;
        int diff = a - b;
        az = __builtin_ctz(diff);
        b = min(a, b);
        a = abs(diff);
    }
    return b << z;
}
vector<vector<int>> get_gcd(int x) {
    int up = 1e3;
    vector<vector<int>> re(up + 1, vector<int>(up
+ 1));
    for (int i = 1; i <= up; i++) {
        for (int j = 1; j <= i; j++) {
            re[i][j] = re[j][i] = fast_gcd(i, j);
        }
    }
    for (int i = 0; i <= up; i++) {
        re[0][i] = re[i][0] = i;
    }
    return re;
}
/*
            auto &[x, y, z] = fact[b[j]];
            int _i = a[i];
            int gcd;
            if (x > 1) {
                gcd = _gcd[_i % x][x];
                tmp = tmp * gcd % mod;
                _i /= gcd;
            }
            if (y > 1) {
                gcd = _gcd[_i % y][y];
                tmp = tmp * gcd % mod;
                _i /= gcd;
            }

            if (z > 1e3) {
                tmp = tmp * (_i % z == 0 ? z : 1) % mod;
            }
            else {
                gcd = _gcd[_i % z][z];
                tmp = tmp * gcd % mod;
            }
*/
```

24. 高精度

（1）版本一（长功能多）

```
struct BigInt {
    static const int BASE = 1e9;
    static const int WIDTH = 9;
    vector<int> s;

    BigInt(long long num = 0) { *this = num; }
    BigInt(string str) { *this = str; }

    BigInt& operator=(long long num) {
        s.clear();
        do { s.push_back(num % BASE); num /=
BASE; } while (num > 0);
        return *this;
    }

    BigInt& operator=(const string& str) {
        s.clear();
        for (int i = str.length(); i > 0; i -= WIDTH) {
            if (i < WIDTH)
s.push_back(stoi(str.substr(0, i)));
            else s.push_back(stoi(str.substr(i -
WIDTH, WIDTH)));
        }
        trim();
        return *this;
    }

    void trim() {
        while (s.size() > 1 && s.back() == 0)
s.pop_back();
    }

    BigInt operator+(const BigInt& b) const {
        BigInt c; c.s.clear();
        for (int i = 0, carry = 0; i < s.size() || i <
b.s.size() || carry; ++i) {
            if (i < s.size()) carry += s[i];
```

```
        if (i < b.s.size()) carry += b.s[i];
        c.s.push_back(carry % BASE);
        carry /= BASE;
    }
    return c;
}

BigInt operator-(const BigInt& b) const {
    BigInt c = *this;
    for (int i = 0, borrow = 0; i < c.s.size(); ++i) {
        borrow = c.s[i] - borrow - (i < b.s.size() ? b.s[i] : 0);
        c.s[i] = borrow < 0 ? borrow + BASE : borrow;
        borrow = borrow < 0 ? 1 : 0;
    }
    c.trim();
    return c;
}

BigInt operator*(const BigInt& b) const {
    BigInt c; c.s.assign(s.size() + b.s.size(), 0);
    for (int i = 0; i < s.size(); ++i) {
        long long carry = 0;
        for (int j = 0; j < b.s.size() || carry; ++j) {
            long long cur = c.s[i + j] + s[i] * 1ll * (j < b.s.size() ? b.s[j] : 0) + carry;
            c.s[i + j] = cur % BASE;
            carry = cur / BASE;
        }
    }
    c.trim();
    return c;
}

bool operator<(const BigInt& b) const {
    if (s.size() != b.s.size()) return s.size() < b.s.size();
    for (int i = s.size() - 1; i >= 0; --i)
        if (s[i] != b.s[i]) return s[i] < b.s[i];
    return false;
}

bool operator==(const BigInt& b) const {
    return !(*this < b) && !(b < *this);
}

friend istream& operator>>(istream& in, BigInt& x) {
    string str; if (in >> str) x = str; return in;
}

friend ostream& operator<<(ostream& out, const BigInt& x) {
    out << x.s.back();
    for (int i = x.s.size() - 2; i >= 0; --i) {
        out << setfill('0') << setw(WIDTH) << x.s[i];
    }
    return out;
}};
```

（2）版本二（较短，功能简单）

```
struct Big {
    vector<int> v;
    const int B = 1e9;

    Big() {} // 默认空向量
    Big(long long n) { do { v.push_back(n % B); n /= B; } while (n); }
    Big(string s) {
        // 利用 max/min 优雅避开边界 if/else 截取判定
        for (int i = s.size(); i > 0; i -= 9)
            v.push_back(stoi(s.substr(max(0, i - 9), min(i, 9))));
        trim();
    }

    void trim() { while (v.size() > 1 && v.back() == 0) v.pop_back(); }

    Big operator+(const Big &b) const {
        Big c;
        // 进位 t 融合进循环条件，省去最后的 if(t) 判定
        for (int i = 0, t = 0; i < v.size() || i < b.v.size() || t; ++i) {
            if (i < v.size()) t += v[i];
            if (i < b.v.size()) t += b.v[i];
            c.v.push_back(t % B);
            t /= B;
        }
        return c;
    }

    Big operator-(const Big &b) const {
        Big c = *this;
        for (int i = 0, t = 0; i < c.v.size(); ++i) {
            t = c.v[i] - t - (i < b.v.size() ? b.v[i] : 0);
            c.v[i] = (t + B) % B;
            t = t < 0; // 核心：t < 0 时自动转为 1 形成借位，否则为 0
        }
        c.trim(); return c;
    }
Big operator*(const Big &b) const {
        Big c; c.v.assign(v.size() + b.v.size(), 0);
        for (int i = 0; i < v.size(); ++i) {
            long long t = 0;
```

```
        // 乘法的进位累加直接一气呵成
        for (int j = 0; j < b.v.size() || t; ++j) {
            t += c.v[i + j] + 1ll * v[i] * (j <
b.v.size() ? b.v[j] : 0);
            c.v[i + j] = t % B;
            t /= B;
        }
    }
    c.trim(); return c;
  }

  bool operator<(const Big &b) const {
    if (v.size() != b.v.size()) return v.size() <
b.v.size();
    for (int i = v.size() - 1; i >= 0; --i)
      if (v[i] != b.v[i]) return v[i] < b.v[i];
    return false;
  }

  friend ostream& operator<<(ostream& os,
const Big& a) {
    if (a.v.empty()) return os << 0;
    os << a.v.back();
    for (int i = a.v.size() - 2; i >= 0; --i)
      os << setfill('0') << setw(9) << a.v[i];
    return os;
  }
};
```

25. 半开半闭区间线段树

主要是文字说明一些注意点，比如左右区间为[l,m]和[m,r]，tl<m，走左，tr>m走右，点是没有长度的，长度由两个点相减得到，叶子节点r-l==1，r-l>=1才算有效区间，在一些区间问题常用这个写法，比如线段长度

26. 杂项

26.1. min - max 恒等式(将 min 问题转化为 max 问题)

对于集合$T \subseteq \{b_1, \ldots, b_{n+m}\}$

$$\min b_i = \sum_{\varnothing \neq T \subseteq \{1,\ldots,n+m\}} (-1)^{|T|-1} \max_{i \in T} b_i$$

26.2. 对区间离线操作的差分处理

```
using ll = long long;

vector<pair<ll, ll>> events;
events.reserve(2 * m + 2);

// 保证 [1,n] 全部被统计，包括覆盖次数为 0
的部分。
events.push_back({1, 0});
events.push_back({n + 1, 0});

for (int i = 0; i < m; ++i) {
  ll l, r;
  cin >> l >> r;
  events.push_back({l, 1});
  events.push_back({r + 1, -1});
}

sort(events.begin(), events.end());

ll active = 0;

for (size_t i = 0; i < events.size(); ) {
  ll L = events[i].first;

  // 同一个位置的变化一起处理。
  while (i < events.size() && events[i].first == L)
{
    active += events[i].second;
    ++i;
  }

  if (i == events.size()) break;

  ll R = events[i].first - 1;

  // 得到一段：[L,R]，每个位置的覆盖次数都
是 active。
  // 在这里处理这一段。
}
```

26.3. 对拍

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
  while (1) //一直循环，直到找到不一样的数
据
  {
    system("data.exe > in.txt");
    system("baoli.exe < in.txt > baoli.txt");
    system("std.exe < in.txt > std.txt");
    if (system("fc std.txt baoli.txt")) //当 fc 返回
1 时，说明这时数据不一样
      break;                  //不一样就跳出循
环
  }
  return 0;
}
//造数据
#include <bits/stdc++.h>
```

```cpp
#include <sys/timeb.h>
int main()
{
    struct _timeb T;
    _ftime(&T);
    srand(T.millitm);

    // freopen("in.txt", "w", stdout); //生成 使两份基本代码 将要读入的数据
    int a = rand(), b = rand();
    printf("%d %d\n", a, b);
}
```

26.4. 一些常见的__builin_函数

```cpp
__builtin_popcount(x) 二进制中 1 的个数
popcount(13) → 3，因为 1101
__builtin_ctz(x) 末尾连续 0 的个数 ctz(12) → 2，因为 1100
__builtin_clz(x) 开头连续 0 的个数 以 unsigned int 位数计算
__builtin_parity(x) 1 的个数是否为奇数 奇数返回 1，偶数返回 0
__builtin_ffs(x) 最低位 1 的位置 从 1 开始编号；x=0 返回 0
// unsigned int
__builtin_popcount(x);
__builtin_ctz(x);
__builtin_clz(x);

// unsigned long
__builtin_popcountl(x);
__builtin_ctzl(x);
__builtin_clzl(x);

// unsigned long long
__builtin_popcountll(x);
__builtin_ctzll(x);
__builtin_clzll(x);
```

26.5. 表达式求值

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

bool isOp(const string& s) {
    return s == "+" || s == "-" || s == "*" || s == "/";
}

ll evalRPN(const vector<string>& tokens) {
    stack<ll> st;

    for (const string& s : tokens) {
        if (!isOp(s)) {
            st.push(stoll(s));
            continue;
        }

        ll b = st.top(); st.pop(); // 右操作数
        ll a = st.top(); st.pop(); // 左操作数

        if (s == "+") st.push(a + b);
        else if (s == "-") st.push(a - b);
        else if (s == "*") st.push(a * b);
        else st.push(a / b);
    }

    return st.top();
}

// 示例：
// cout << evalRPN({"3", "4", "+", "5", "*", "6", "2", "/", "-"});
// 输出：32

//支持非负整数、二元 + - * /、括号、空白；不支持 -3、2*(-3) 这类一元负号。
int priority(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

vector<string> toRPN(const string& s) {
    vector<string> out;
    stack<char> ops;
    int n = (int)s.size();

    for (int i = 0; i < n; ) {
        char c = s[i];

        if (isspace((unsigned char)c)) {
            ++i;
        } else if (isdigit((unsigned char)c)) {
            // 连续数字要作为一个整体输出
            int j = i;
            while (i < n && isdigit((unsigned char)s[i])) {
                ++i;
            }
            out.push_back(s.substr(j, i - j));
        } else if (c == '(') {
            ops.push(c);
            ++i;
        } else if (c == ')') {
            while (ops.top() != '(') {
                out.push_back(string(1, ops.top()));
                ops.pop();
            }
```

```
            ops.pop(); // 丢弃左括号
            ++i;
        } else {
            while (!ops.empty() && ops.top() != '(' &&
                   priority(ops.top()) >= priority(c)) {
                out.push_back(string(1, ops.top()));
                ops.pop();
            }
            ops.push(c);
            ++i;
        }
    }

    while (!ops.empty()) {
        out.push_back(string(1, ops.top()));
        ops.pop();
    }

    return out;
}

//如何使用
int main() {
    string s;
    getline(cin, s);

    auto tokens = toRPN(s);

    for (const auto& t : tokens) cout << t << ' ';
    cout << '\n' << evalRPN(tokens) << '\n';
}
```

26.6. 优先队列自定义结构体比较

```
struct cmp{
    bool operator() (const node &x, const node &y)
{
        //return x.a * y.b > x.b * y.a;
        return x.a * y.b < x.b * y.a;
    }
}; // 队首元素 a * other.b > b * other.a,这个细节需要注意,是反过来的
priority_queue<node, vector<node>, cmp> q;
```

26.7. __int128 输出(cout 实现)

```
void print(__int128 x) {
    if (x == 0) {
        cout << "0\n";
        return;
    }
    string s;
    while (x) {
        s.push_back(char('0' + x % 10));
        x /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s << '\n';
}
```

26.8. 三分

```
//浮点数
#include <bits/stdc++.h>
using namespace std;

double f(double x) {
    return (x - 3) * (x - 3) + 2; // 替换成题目函数
}

// 返回最小值对应的位置 x
double ternary(double l, double r) {
    for (int i = 0; i < 100; ++i) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;

        if (f(m1) < f(m2))
            r = m2;
        else
            l = m1;
    }
    return l + (r - l) / 2;
}

int main() {
    double x = ternary(-100, 100);
    cout << fixed << setprecision(10);
    cout << x << '\n';   // 最优位置：3
    cout << f(x) << '\n'; // 最小值：2
}

//整数
using ll = long long;

ll f(ll x) {
    return (x - 3) * (x - 3) + 2; // 注意溢出
}

// 返回最小值对应的一个整数位置 x
ll ternary(ll l, ll r) {
    while (r - l > 3) {
        ll m1 = l + (r - l) / 3;
        ll m2 = r - (r - l) / 3;

        if (f(m1) < f(m2))
            r = m2 - 1;
        else
            l = m1 + 1;
    }
```

```
    ll ans = l;
    ll best = f(l);
    for (ll x = l + 1; x <= r; ++x) {
        ll val = f(x);
        if (val < best) {
            best = val;
            ans = x;
        }
    }
    return ans;
}
```

26.9. GF(2)高斯消元判矩阵满秩

区间 01 向量 / GF(2) / 图 / DSU 判满秩

1. GF(2)
   - 数只有 0、1
   - 加法、减法都等于 xor
   - 线性组合 = 任选若干向量 xor
   - 存在非空向量集合 xor 后为 0
     <==> 线性相关
     <==> 矩阵不满秩

2. 区间差分
   原区间 [l, r] 的行向量：
   0 ... 0 1 1 1 ... 1 0 ... 0

   对每行做 GF(2) 差分：
   d[i] = a[i] xor a[i - 1]
   并补一个 n+1 位置。

   则区间 [l, r] 只在 l、r+1 两处为 1：
   [l, r] -> {l, r+1}

   差分可以通过前缀 xor 还原，属于可逆变换，因此不改变向量组是否线性相关 / 矩阵是否满秩。

3. 建图
   把位置 1...n+1 看成图的点：
   区间 [l, r] -> 一条边 (l, r+1)

   差分后的每一行都恰好是这条边的关联向量：
   仅在边的两个端点处为 1。

4. 为什么“线性相关 <==> 有环”
   - 若有环：环上所有边 xor。
     每个点在环上恰好出现两次，1 xor 1 = 0，
     所以结果为全 0，边向量线性相关。

   - 若无环：图是森林，必有叶子。
     叶子只属于一条边；若若干边 xor 为 0，
     那条边必须不选。不断删除叶子，所有边都不能选，
     因此边向量线性无关。

   所以：
   边向量线性无关 <==> 图无环
   边向量线性相关 <==> 图有环

5. DSU = 此特殊矩阵的高斯消元
   加边 (u, v) 时：
   find(u) == find(v)
   => 已有 u 到 v 的路径
   => 新边 + 路径形成环
   => 新行 = 路径上旧行的 xor
   => 新行线性相关，秩不增加

   否则 unite(u, v)，新边独立，秩增加 1。

6. 本题
   n 个区间 -> n 条边；
   点数为 n+1。

   所有行满秩
   <==> n 条边线性无关
   <==> 图无环
   <==> n+1 个点、n 条无环边
   <==> 图是一棵树。

   实现：
   for each [l, r]:
   if (find(l) == find(r + 1)) ans = 0;
   else unite(l, r + 1);

26.10. sos dp

```
/*
   做n次一维前缀和
*/
for(int i[1]=1;i[1]<=n;i[1]++)
   for(int i[2]=1;i[2]<=n;i[2]++)
      ...
      for(int i[n]=1;i[n]<=n;i[n]++)
         a[i[1]][i[2]][i[3]]...+=a[i[1]-1][i[2]]...;
for(int i[1]=1;i[1]<=n;i[1]++)
   for(int i[2]=1;i[2]<=n;i[2]++)
      ...
      for(int i[n]=1;i[n]<=n;i[n]++)
         a[i[1]][i[2]][i[3]]...+=a[i[1]][i[2]-1]...;
...
```

26.11. CDQ 分治

```
/*
在数组中找到一个最大的下标集合,使得
x[i],y[i]均为不下降序列,求出最大大小即可

通常用在以下情况
```

```
dp[i] = 从 j<i 的状态中，按若干数值条件筛选后取最值/求和

尤其是：

状态有明确的时间或下标先后关系；
转移可以离线处理；
一个维度适合分治，其他维度能用 BIT、线段树或其他数据结构维护。
*/
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

using namespace std;

class FenwickMax {
  int size_;
  vector<int> tree_;
  vector<int> touched_;

public:
  explicit FenwickMax(int size) : size_(size), tree_(size + 1, 0) {
    touched_.reserve(size);
  }

  void update(int position, int value) {
    for (int p = position; p <= size_; p += p & -p) {
      if (tree_[p] == 0) touched_.push_back(p);
      tree_[p] = max(tree_[p], value);
    }
  }

  int query(int position) const {
    int result = 0;
    for (int p = position; p > 0; p -= p & -p) {
      result = max(result, tree_[p]);
    }
    return result;
  }

  void clear() {
    for (int p : touched_) tree_[p] = 0;
    touched_.clear();
  }
};

class CDQSolver {
  int n_;
  vector<int> x_, y_, dp_, order_, temp_;
  FenwickMax bit_;

  bool byXThenId(int a, int b) const {
    if (x_[a] != x_[b]) return x_[a] < x_[b];
    return a < b;
  }

  // 进入和返回时，order_[l..r] 都按 (x, 下标) 排序。
  void solve(int l, int r) {
    if (l == r) return;
    int mid = (l + r) / 2;

    // 按原下标把已按 x 排序的序列稳定分成左右两组。
    int left = l;
    int right = mid + 1;
    for (int k = l; k <= r; ++k) {
      int id = order_[k];
      if (id <= mid) temp_[left++] = id;
      else temp_[right++] = id;
    }
    copy(temp_.begin() + l, temp_.begin() + r + 1, order_.begin() + l);

    solve(l, mid);

    // 左半 dp 已经最终确定；按 x 扫描，BIT 维护 y 前缀最大 dp。
    int p = l;
    for (int q = mid + 1; q <= r; ++q) {
      int current = order_[q];
      while (p <= mid && x_[order_[p]] <= x_[current]) {
        int previous = order_[p++];
        bit_.update(y_[previous], dp_[previous]);
      }
      int best = bit_.query(y_[current]);
      if (best > 0) dp_[current] = max(dp_[current], best + 1);
    }
    bit_.clear();

    solve(mid + 1, r);

    // 归并两个已按 x 排序的子区间，恢复当前区间的排序。
    int a = l, b = mid + 1, k = l;
    while (a <= mid && b <= r) {
      if (byXThenId(order_[a], order_[b])) temp_[k++] = order_[a++];
      else temp_[k++] = order_[b++];
    }
    while (a <= mid) temp_[k++] = order_[a++];
    while (b <= r) temp_[k++] = order_[b++];
    copy(temp_.begin() + l, temp_.begin() + r + 1, order_.begin() + l);
  }
```

```
public:
    CDQSolver(vector<int> x, vector<int> y)
        : n_(static_cast<int>(x.size()) - 1),
x_(move(x)), y_(move(y)),
          dp_(n_ + 1, 1), order_(n_ + 1), temp_(n_ +
1), bit_(n_) {
        iota(order_.begin(), order_.end(), 0);
        sort(order_.begin() + 1, order_.end(),
            [&](int a, int b) { return byXThenId(a,
b); });
        // y 已在 main 中压缩到 [1,n]，dp[0] 不参
与转移。
        dp_[0] = 0;
    }

    int answer() {
        if (n_ == 0) return 0;
        solve(1, n_);
        return *max_element(dp_.begin() + 1,
dp_.end());
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> x(n + 1), y(n + 1), allY;
    allY.reserve(n);
    for (int i = 1; i <= n; ++i) {
        cin >> x[i] >> y[i];
        allY.push_back(y[i]);
    }

    sort(allY.begin(), allY.end());
    allY.erase(unique(allY.begin(), allY.end()),
allY.end());
    for (int i = 1; i <= n; ++i) {
        y[i] =
static_cast<int>(lower_bound(allY.begin(),
allY.end(), y[i]) - allY.begin()) + 1;
    }

    // BIT 的大小需要覆盖压缩后的 y 值；n 更
大时仍然足够。
    CDQSolver solver(move(x), move(y));
    cout << solver.answer() << '\n';
    return 0;
}
```

26.12. pb_ds

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

template<class T>
using ordered_set = tree<
    T,
    null_type,
    less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update
>;
/*
核心函数
s.order_of_key(x);  // 小于 x 的元素个数
s.find_by_order(k); // 第 k 小元素的迭代器，k
从 0 开始
*/
void s1() {
    ordered_set<int> s;

    s.insert(5);
    s.insert(2);
    s.insert(9);

    cout << s.order_of_key(5) << '\n';     // 1，只
有 2 小于 5
    cout << *s.find_by_order(0) << '\n';   // 2
    cout << *s.find_by_order(1) << '\n';   // 5
    cout << *s.find_by_order(2) << '\n';   // 9

    s.erase(5);
}
/*
如果题目要处理重复元素，推荐用 pair<int,
int>
*/
ordered_set<pair<int, int>> s;
int idx = 0;

void insert(int x) {
    s.insert({x, ++idx});  //这样每个相同的 x 会因
为 idx 不同而被当成不同元素。
}

//一些常见操作的模板
const int INF = 1e9;

ordered_set<pair<int, int>> s;
int idx = 0;
```

```cpp
void insert(int x) {
    s.insert({x, ++idx});
}

// 删除一个 x
void erase_one(int x) {
    auto it = s.lower_bound({x, -INF});
    if (it != s.end() && it->first == x) {
        s.erase(it);
    }
}

// x 的排名：比 x 小的数 + 1
int rank_of(int x) {
    return s.order_of_key({x, -INF}) + 1;
}

// 第 k 小，k 从 1 开始
int kth(int k) {
    return s.find_by_order(k - 1)->first;
}

// 严格小于 x 的最大值
int predecessor(int x) {
    int pos = s.order_of_key({x, -INF});
    return s.find_by_order(pos - 1)->first;
}

// 严格大于 x 的最小值
int successor(int x) {
    auto it = s.upper_bound({x, INF});
    return it->first;
}
/*
一些注意事项
find_by_order(k) 的 k 是 从 0 开始。
order_of_key(x) 返回的是 严格小于 x 的数量。
PBDS 不是标准 C++，只能在 GNU G++ 下用。
不建议用 less_equal<int> 来模拟 multiset，删除和查找容易出怪问题；打 ACM 用 pair<int,int> 最稳。
*/
```

26.13. floyd 判圈法

Floyd 判圈算法使用两个指针：
慢指针（Tortoise）：每次移动一步。
快指针（Hare）：每次移动两步。
如果链表中存在环，那么快指针和慢指针最终会在环中相遇。如果链表中不存在环，快指针会先到达链表的末端。

```cpp
// Floyd 判圈法（快慢指针）
// 适用于链表或函数图：每个节点至多有一个后继节点
template <class Next>
bool floydCycle(int start, Next next, int nullNode = -1) {
    int slow = start;
    int fast = start;

    while (fast != nullNode && next(fast) != nullNode) {
        slow = next(slow);        // 每次走一步
        fast = next(next(fast));  // 每次走两步

        if (slow == fast) {
            return true;
        }
    }

    return false;
}
```

```cpp
//求环入口+长度
struct CycleInfo {
    bool hasCycle;
    int entry;  // 环入口
    int length; // 环长度
};

template <class Next>
CycleInfo findCycle(int start, Next next, int nullNode = -1) {
    int slow = start, fast = start;

    // 1. 判断是否有环
    do {
        if (fast == nullNode || next(fast) == nullNode) {
            return {false, nullNode, 0};
        }
        slow = next(slow);
        fast = next(next(fast));
    } while (slow != fast);

    // 2. 求环入口
    slow = start;
    while (slow != fast) {
        slow = next(slow);
        fast = next(fast);
    }
    int entry = slow;

    // 3. 求环长度
    int length = 1;
    fast = next(entry);
    while (fast != entry) {
        fast = next(fast);
        ++length;
```

```
    }

    return {true, entry, length};
}
```

26.14. st 表上二分(待施工)

26.15. 向上取整与向下取整

```
auto floor_div = [](i64 x, i64 y) -> i64 {
    i64 q = x / y, r = x % y;
    return q - (r != 0 && ((r > 0) != (y > 0)));
};

auto ceil_div = [](i64 x, i64 y) -> i64 {
    i64 q = x / y, r = x % y;
    return q + (r != 0 && ((r > 0) == (y > 0)));
};
```

26.16. 随手记

1.upper_bound和lower_bound比map更快
2.move函数和merge函数的用法
3.#define File(name) freopen(#name".in", "r", stdin); freopen(#name".out", "w", stdout);
4.x &= x - 1 每次删去最低位置上的1, Sub = (sub − 1) & mask // 遍历该集合的所有子集
5. >> 优先级比 & 高
6.一些并行运算确实要更快
7.Prüfer序列(造树)
8.负数的向上取整不太一样(-x + mod - 1) / mod

```
i64 ceil_div(i64 a, i64 b) {
    if (a >= 0) return (a + b - 1) / b;
    return a / b;
}
```

9.对于整数随机变量:
E[M]=\sum_{x >= 1}\Pr(M >= x) = \sum_{x >= 1} x * p(x)
右边可以化为左边的试试
10.vector.assign的复杂度是o(n),vector.clear()会把size清空导致访问越界
11.树状数组加入0下标会死循环
12.floyd判环的快慢指针,一快一慢的技巧很常见,比如可以差一来求相邻的差
13.move的移动赋值效率比简单复制要高 <utility>
比如 cnt = std::move(next_cnt); 把一整个容器赋给cnt,比cnt = next_cnt快,注意这样之后不要访问next_cnt的内容了,移动后里面的值可能会有改变
14.next函数,获取下一个位置的迭代器, 比如current = std::next(pre); 直观+好用 <iterator>
15.dsu判环
16.ull 会自动截掉运算结果超出 64 位的部分,但不允许移位次数达到或超过64(左移的位数>=字长或移负数,在C中是未定义行为,不要依赖结果)
17.浮点数二分固定次数效果比r-l>=eps更好,具体次数为 L/2^k <= eps ,即得到 k>= [log2(L/eps)](向上取整),通常多算几次,如果出现mid == l or mid == r,提前break
18.eps 不是浮点题的必需品,更不能在所有比较里统一加一个 eps,别瞎用,可以了解下浮点数的存储方式
19.auto x = lower_bound(c_set.begin(), c_set.end(), i);
对 multiset 使用通用的 std::lower_bound, 时间复杂度是(O(n)), 不是 (O(log n))
虽然比较次数只有 (O(log n)), 但 multiset 的迭代器不能随机跳转, 寻找中点需要逐步移动, 总共会移动 (O(n)) 次。
用multiset的成员函数即可,auto x = c_set.lower_bound(i); (O(log n))
20.std::sort 要求比较函数满足严格弱序 比如 a > b是合法的 a >= b就是未定义的

经典结论：通过区间 +1 操作将全零序列变为序列 c_i，需要的花费为 $\sum \max(0, c_i - c_{i-1})$。

由此可以得到：

$$\text{ans} = \sum \max(0, b_i - b_{i-1})$$

26.17. 碎碎念

1.若
a[x1]=...
b[x2]=a[x3]+...
最好开新变量存旧值a，可以避免一些不好发现的错误

26.18. 一些思维(废话)

1.由少到多,模拟过程找性质
找下一个
从条件入手,紧抓条件,找方向
2.树上选点,如果可以重复的话,对称可能可以产生一些性质,可以尝试平均等方法利用性质
3.换根问题转化为固定根
4.贪心题从收益函数的角度去分析
5.树的问题可以联系子树大小去考虑,或者有时侯树链剖分/dsu on tree,图可能是染色,度数,连边可能边有选择方向的含义(有向边),或者是一些基础量,像桥之类的,数论+树结合的题目也有(数论(莫反)得出要维护的东西后dsu on tree的做过),图也可以去掉一些边变成树

6.每个点度数大等于2,这个图不存在链
7.抓住题目不变量
8.从一到多比从多到一或许处理起来更简单

待施工： fwt，博弈论，根号分治，调和级数，点分治，polya 定理带权重的版本，猫树，无旋 treap，splay 树，区间 gcd 最多下降 log 次，斐波那契数列的性质应用（每项大等于前一项，每一项小等于前一项的两倍，每一项等于前两项的和），重心点分治每次规模除二，st 表二分，对顶堆，isap 被特意卡的话常数比 dinic 大，

// 1. (x&b)>(b>>1) 判断 x 在 b 的最高位是否为 1

// a + b == (a ^ b) + 2 * (a&b)

std::set：通过 Key、Compare、Allocator 三个参数，定义键类型、排序规则和内存分配。

std::map：在 set 的基础上增加 T 值类型，形成键、值、比较器、分配器四个可调参数。

std::unordered_set：用 Hash、KeyEqual 替代 Compare，组合成键、哈希、判等、分配器四个参数的无序集合。

std::unordered_map：在 unordered_set 模板参数上增加 T 值类型，成为键、值、哈希、判等、分配器五个可调参数的无序映射。

Bfs trick : Problem - B - Codeforces（来自 cc cpc）

四个边框加入队列中 bfs，可方便解决左上角和右下角是否连通等问题（本题左边框和下边框作为起点 bfs，扫八个方向，标记为 1，右边框与上边框作为起点，扫八个方向，标记为 2，如果 1 与 2 连接（第二次八方扫描时扫描到），则左上与右下被隔绝）

当前树平均深度为 Am，则加入一点作为原图上点的新儿子的期望深度为 Am + 1,有时候期望计算可以从平均角度考虑

二进制分组

根据经典结论全零序列区间 得到序列 需要花费

数与数之间的关系问题转化为图上问题

矩阵乘法优化 dp 问题

lcp 优化--Ukkonen 算法或 Myers 差分算法

期望 dp，计数 dp（插入 dp），动态 dp，状压 dp

笛卡尔树，差分约束，猫树，折半搜索，判断能否构成立体图形计算某点所有距离平方